

Colorado State University Pueblo

School of Engineering

A Graph Neural Surrogate for Longitudinal Train Dynamics: Toward Fuel-Optimal Control of Freight Consists

George James

August 18, 2026

Abstract

Freight-train longitudinal dynamics (LTD) simulators capture nonlinear slack couplers, per-vehicle grade and curvature, actuator lag, and power-limited traction with high fidelity, but their computational cost — tens to hundreds of CPU-hours per scenario — makes them impractical for the repeated evaluation that control design and operational planning require. This thesis develops a fast, train-agnostic surrogate of longitudinal train dynamics that serves as a differentiable simulation backbone for fuel-optimal control of freight consists.

The consist is represented as a chain graph, with vehicles as nodes and couplers as edges, using a tensorized node/edge state $(\mathbf{H}, \mathbf{E})$ that generalizes across arbitrary train length without padding or masking. A message-passing graph neural network learns a physics-residual correction to a known force-balance right-hand side, integrated as a neural ODE, so predictions inherit the consist’s chain topology and remain differentiable end to end for downstream control. This design targets structural generalization: models trained on one range of consist sizes and route conditions should extrapolate to unseen train lengths and corridors, rather than relying on the fixed-length token handling that sequence models require.

Training data are produced by a reference LTD simulator, developed and validated stage by stage (single-vehicle traction, linear and nonlinear slack couplers, N -vehicle assemblies, per-vehicle route sampling, and actuator/power-limited traction), driven along real-terrain route profiles derived from elevation and alignment data. The surrogate is evaluated against physics-only and physics-residual MLP baselines, and against a transformer variant demoted to an ablation, on trajectory fidelity, trip-level summary accuracy (energy, arrival time, peak coupler force), and generalization across held-out consist sizes, grades, and corridors.

The trained surrogate then serves as the world model for a fuel-optimal control demonstration: minimizing trip energy on held-out real routes subject to speed limits and coupler-force safety constraints, using model-predictive control that exploits the surrogate’s analytic gradients. Throughout, we are explicit that simulator parameters are illustrative rather than calibrated to a specific real train, so the claims concern fidelity to the reference simulator, structural generalization across consist size and route, and control performance under that model — not fidelity to real rolling stock.

Acknowledgments

Contents

Abstract	3
Acknowledgments	5
1 Introduction and Problem Motivation	17
1.1 Background: Longitudinal Train Dynamics	17
1.2 Surrogate Modeling	17
1.3 Thesis Objective and Contributions	18
1.4 Organization of Thesis	18
2 Related Work	19
2.1 Classical Train Longitudinal Dynamics and Simulation	19
2.1.1 Modeling Abstractions and Levels of Fidelity	19
2.1.2 Computational Schemes and Numerical Challenges	20
2.1.3 Benchmarking, Applications, and Limitations	20
2.2 Data-Driven and Hybrid Modeling in Rail	21
2.2.1 Surrogate Models for Accelerating Physics-Based Simulation	21
2.2.2 Data-Driven Estimation of In-Train Forces	22
2.2.3 Surrogate Modeling for Operations and Optimization	22
2.2.4 Limitations of Existing Data-Driven and Hybrid Approaches	23
2.2.5 Implications for Hybrid Probabilistic Modeling	23
2.3 Transformers for Dynamical Systems and Spatiotemporal Forecasting	23
2.3.1 Long-horizon Forecasting Transformers	24
2.3.2 Neural Operators for PDE-governed Dynamics	25
2.3.3 Implications for Train Dynamics and Spatiotemporal Decision Support	26
2.4 Probabilistic Forecasting and Uncertainty Calibration	26
2.4.1 Foundations of Probabilistic Regression	27
2.4.2 Empirical Miscalibration in Deep Models	29
2.4.3 Post-hoc Calibration for Regression	29
2.4.4 Distribution-Free Coverage via Conformal Prediction	31
2.4.5 Proper Scoring Rules and Evaluation Metrics	31
3 Problem Formulation	33
3.1 The prediction and control problem, and the tensor format used to describe it	33
3.1.1 Physical setting and prediction task	33

3.1.2	Classical longitudinal dynamics formulation	34
3.1.3	Tensorized state representation	35
3.1.4	Dynamics in tensor-oriented form	35
3.1.5	Forecasting objective and simulator operator	36
3.2	Input definitions	37
3.2.1	Route profile	37
3.2.2	Consist configuration	37
3.2.3	Control plan	37
3.3	Output definitions	38
3.3.1	Trajectory-level outputs	38
3.3.2	Aggregate outputs	38
3.3.3	Risk metrics	38
3.4	Uncertainty sources	39
3.4.1	Formal treatment of uncertainty	39
3.5	Supervised Learning Objective	39
3.5.1	Loss term definitions	40
3.6	Conceptual pipeline	40
3.7	Evaluation targets	41
3.8	Chapter summary	41
4	Reference Simulator	43
4.1	Stage 1: single powered vehicle	43
4.2	Stage 2: locomotive and one car with a linear coupler	45
4.3	Stage 3: nonlinear coupler with slack and asymmetric buff/draft	45
4.4	Stage 4: three bodies and sequential coupler forces	45
4.5	Stage 5: general N -vehicle assembly	49
4.6	Stage 6: per-vehicle grade and curvature from a route profile	51
4.7	Stage 7: actuator dynamics and power-limited traction	52
4.8	Stage 8: scope relative to industrial high fidelity	52
4.9	Chapter summary	54
5	Route Acquisition and Terrain Data Pipeline	55
5.1	Stage 1: Acquisition	55
5.1.1	Why acquisition has to be area-first	55
5.1.2	Corridor extraction as a graph problem	56
5.1.3	Putting fragments back together	57
5.1.4	Interactive selection	57
5.2	Stage 1.5: Resampling to a uniform grid	58
5.3	Stage 2: Elevation	60
5.4	Stage 3: Deriving the route profile	60
5.4.1	Correcting for bridges and tunnels	60
5.4.2	Smoothing and differentiating together	61
5.4.3	Speed limit and output	61
5.5	Export	63
5.6	Keeping large queries from failing	63

5.7	Two ways to build a dataset	63
5.8	Testing and design choices	63
5.9	Chapter summary	64
6	RailLab: A Desktop Research Console	65
6.1	Why a graphical console	65
6.2	Architecture	66
6.3	The workflow, screen by screen	66
6.3.1	Route	66
6.3.2	Consist	67
6.3.3	Controls	67
6.3.4	Simulate and Results	70
6.4	Status and limitations	70
6.5	Chapter summary	70

List of Figures

2.1	Quantile calibration: diagonal = perfect; uncalibrated model deviates; isotonic calibration corrects.	30
3.1	Conceptual pipeline from structured inputs to tensorized simulator outputs, surrogate forecasting, and fuel-optimal control.	41
4.1	Stage 1 reference run: single vehicle on level track with a smooth traction ramp. Top to bottom: position, speed, and net longitudinal force $F_{\text{trac}} - R(v)$ versus time.	44
4.2	Stage 2: two-body train with linear coupler. Speeds of lead and trailing vehicle (overlapping at this scale), coupler force on the rear car, and extension δ . . .	46
4.3	Stage 3: comparison of linear spring-damper force (Stage 2 parameters) versus nonlinear slack coupler force along the same trajectory, with extension and relative speed.	47
4.4	Stage 4: three-body train with nonlinear couplers. Speeds, coupler forces F_0 and F_1 , and end-to-end stretch versus time.	48
4.5	Stage 4 optional scenario: lead-only brake pulse; speeds and coupler forces. .	49
4.6	Stage 5: N -vehicle train; selected speeds and coupler forces versus time. . . .	50
4.7	Stage 5: coupler force heatmap (force versus time and coupler index).	50
4.8	Stage 6: interpolated grade profile $\sin \theta$ versus route position, with lead and rear speeds and coupler forces versus simulation time.	51
4.9	Stage 7: lead speed with brake-command onset marker; first-order lag states z_{brk} and z_{trac} on the lead versus time.	53
5.1	Overview of the route acquisition pipeline, from raw OpenStreetMap track data to the route-tensor fields consumed by the reference simulator.	55
5.2	Railway mapping around Pueblo, Colorado, rendered from OpenStreetMap track data. Mainline corridors (orange) run through a dense tangle of yard and service track (brown) and meet at several junctions. There is no single map object corresponding to “the line through Pueblo”; corridors have to be recovered from the segments.	56

5.3	The acquisition panel of the route application. A selection mode (radius, bounding box, or relation identifier) is chosen on the right; the centre point, radius, admissible railway types, and a minimum corridor length are set on the left. The mainline-only filter excludes yards and spurs, and the bounded outward crawl described in section 5.1 is toggled at the bottom. Several Overpass endpoints are selectable because any one of them may be rate-limiting at a given moment.	58
5.4	Inspection view for a resampled route: a 49.1 km corridor rewritten onto a uniform 10 m grid, yielding 4920 points. The plan view distinguishes points that sit on an original map vertex from those interpolated across long source segments (713 of them here), so stretches where the geometry is inferred rather than mapped remain visible downstream.	59
5.5	An elevated route rendered in three dimensions, coloured by elevation along the line, with the mapped ground track shown alongside. Visual inspection at this stage catches corridors that have been stitched incorrectly or that jump between parallel alignments, before any derivative is taken.	60
5.6	Ground elevation against arc length for the same corridor, climbing from roughly 1430 m to 1740 m over 49 km. This is the raw signal from which grade is derived; the metre-scale error it carries is negligible against the elevation range plotted here but is large against the elevation <i>change</i> between adjacent 10 m samples, which is what motivates the smoothing described below.	61
5.7	The three derived fields the simulator consumes, plotted against chainage in kilometres: grade $\sin \theta(s)$ as a percentage (top), the curvature proxy $\kappa(s)$ (bottom left), and the speed limit $v_{\max}(s)$ (bottom right). The noisier grade over the first 20 km is the urban approach, where frequent bridges, junctions, and short mapped segments degrade the elevation signal; beyond it the line settles onto a sustained climb near one percent. The speed limit is a step field, here rising where the line leaves restricted urban trackage.	62
6.1	RailLab’s layering. Screens call services; services are the only modules that import the numerical backend, and they return plain data. The backend packages are unchanged by the application’s existence and continue to run headless.	66
6.2	The CONSIST screen: a library of reusable car types (left), the ordered train under construction with per-car overrides (centre), and saved trains (right). The running summary reports car count, total mass, total length, and the number of powered vehicles.	68
6.3	The CONTROLS screen: randomization bounds and distributed-power settings on the left, with a live preview of the generated traction command per power role (top right) and the shared brake command (bottom right). The offset between the three traction traces is the configured lag of the mid-train and rear helper units behind the head end.	69

- 6.4 The RESULTS screen after a 120s run of a 126-vehicle consist under a randomized control profile. Every vehicle's speed is plotted above every coupler's force, on time-linked axes. The information is carried by the spread between traces: the widening and collapsing speed envelope is slack running in and out, with the corresponding fan of coupler forces below. 71

List of Tables

1.1	Reported computational cost of physics-based rail simulation, and the reduction achieved by a learned surrogate. Figures are quoted as reported by the cited studies; they are not normalized to a common hardware baseline. . . .	17
2.1	Representative longitudinal train dynamics (LTD) simulation tools	21

Chapter 1

Introduction and Problem Motivation

1.1 Background: Longitudinal Train Dynamics

Longitudinal train dynamics (LTD) concern the motion and force distribution through the length of a train. A freight train is a distributed mechanical system, often consisting of tens to hundreds of individual coupled masses. The system involves complex and highly non-linear interactions between locomotives, freight cars, and the track profile. Modern simulations of these system interactions rely on detailed physical models which require solving differential equations for each vehicle, resulting in high computational cost. Rapid scenario exploration can be impossible due to the computational resource demand.

Task	Reported cost	Source
Whole-trip LTD optimization study, sequential execution	~18 months	[1]
Same study, parallel execution across multiple processors	11 days	[1]
Coupler-force evaluation, share of LTD solver runtime	~65%	[2]
Multibody vehicle-track simulation of a 5 km segment	~30 minutes	[3]
Learned surrogate for the same 5 km segment	~8 seconds	[3]

Table 1.1: Reported computational cost of physics-based rail simulation, and the reduction achieved by a learned surrogate. Figures are quoted as reported by the cited studies; they are not normalized to a common hardware baseline.

1.2 Surrogate Modeling

High-fidelity physics simulators are essential for understanding complex physical systems, however the computational expense of using such simulators can put serious speed limits on evaluation. This is the case especially in modern analysis or control problems, where complex simulations can require hundreds to thousands of hours of CPU time to simulate multi-body systems. These models are invaluable for theory validation, but they are completely impractical when there exists a need for rapidly repeated scenario evaluation.

Surrogate modeling addresses this problem by learning an approximate mapping of the inputs and system response outputs. This learned model retains the underlying physics at an approximated level, but more importantly can reduce evaluation cost by several orders of magnitude. Trained surrogate models can typically output an accurate prediction in milliseconds or less. This enables rapid simulation of a complex system that would normally consume many CPU hours.

Surrogate models do not replace deterministic physics models, but rather complement them. High-fidelity simulations allow for the data generation needed to build a surrogate model, and also serve as the physical grounding which couples with the surrogate to produce a rapid yet physically informed simulation environment.

1.3 Thesis Objective and Contributions

This thesis presents a physics-transformer hybrid architecture that combines deterministic, reduced-order physics models of the longitudinal train dynamics with a data-driven transformer model. This architecture will enable rapid scenario evaluation while retaining physical constraints.

1. A modular schema that integrates physics-based feature generation with transformer-based scenario modeling
2. Probabilistic output formulation representing quantile regression and exceedance probability estimation
3. Validation methodology demonstrating improved accuracy and reduced computation load compared to physics-only baselines
4. Ablation studies analyzing the contribution of physics integration and attention factorization

1.4 Organization of Thesis

This thesis is organized as follows. Chapter 2 reviews related work in train dynamics, surrogate modeling, and transformer architectures. Chapter 3 formulates the problem mathematically, defining inputs, outputs, and uncertainty sources. ?? presents the framework architecture, including the physics layer, transformer surrogate, and hybrid integration strategy. ?? describes data generation and training methodology. ?? presents evaluation results and validation studies. ?? discusses implications and limitations. ?? concludes with future work directions.

Chapter 2

Related Work

2.1 Classical Train Longitudinal Dynamics and Simulation

Longitudinal train dynamics (LTD) is the study of the physical interactions within a connected train body comprised of multiple linked masses, henceforth referred to as a *consist*. Classical longitudinal train dynamics is defined as the motion of rolling stock vehicles in the longitudinal dimension of the track, ignoring lateral or vertical forces in most cases. The motion of the whole train as well as the individual motions of each vehicle are accounted for in this schema [4].

Early attempts at LTD study treated a train as a continuous elastic rod with a traction force applied at one end, and enabled the study of longitudinal vibration phenomena. This later progressed to discretized mass-spring chains. Early models such as these generally ignored discontinuities, such as those arising from coupler slack, which enabled closed form solutions that required no numerical solver. Nonlinearities were identified early, such as draft gear behavior, but could not be simulated with the level of computing power at the time [5].

2.1.1 Modeling Abstractions and Levels of Fidelity

LTD can be framed as layered abstractions with increasing levels of complexity. At the greatest order-reduction level, LTD can be modeled as point mass or a single-body train model. This level of complexity can be useful for modeling energy usage or scheduling, but is highly limited in the observability of the system, i.e. many variables are unaccounted for. At the next level of abstraction, the consist can be modeled as a multi-mass system, where each vehicle is accounted for as a rigid body with one longitudinal degree of freedom. Masses are connected with modeled couplers and draft gears, which introduce nonlinear interactions. Beyond this, we can extend the DOF of the model to account for motions in the remaining spatial dimensions, which allows for observability track curve effects, in train instabilities, and even derailment scenarios; such models are formulated using the general multibody dynamics machinery of constrained rigid and flexible bodies [6]. This scale of modeling naturally trades speed for fidelity, as computation of so many parameters is highly expensive, but the high fidelity is necessary for some analyses [5].

2.1.2 Computational Schemes and Numerical Challenges

The standard computation schema for LTD treats the problem as an repeated evaluation of all force elements followed by state propagation [2]. In general, a simulator first identifies force elements (propulsion resistance, curving resistance, grade components, gravity, traction forces, dynamic brake forces, air brake forces, and coupler forces). Then, the system computes each element from the inputs (time, vehicle velocities, accelerations, and vehicle locations). Finally, velocities and positions are updated through a numerical solver. The clear modularity of this computing system allows for addition or replacement of new computing models, such as replacing a module with a learned surrogate model.

The critical difficulty with computing LTD is with the vehicle connection systems, specifically the couplers, draft gears, and drawbars. Friction draft gears exhibit extreme nonlinearities, including hysteresis and stick-slip) that are hard to both model and numerically solve [5]. The coupler force calculation accounted for approximately 65% of the computing time in the study described in [2]. The difficulty of handling this interaction further motivates the effectiveness of the hybrid physics and transformer surrogate model, which is focused on handling the hard parts of the model while keeping the backbone physics modeling intact.

2.1.3 Benchmarking, Applications, and Limitations

LTD has historically lacked standardized procedures for benchmarking LTD simulators, which limits objective comparison of tools developed by different institutions. Spiryagin et. al. introduced an international benchmarking initiative to define a mutual set of input scenarios, operating conditions, and necessary outputs to compare efficacy of various simulators [7]. The benchmarking initiative was motivated by the fact that most LTD simulation tools have evolved independently as in-house software for the many institutions that use them. This resulted in limited to no transparency in terms of numerical methods, assumptions, and performance. Participating simulators used identical train configurations, commands for traction and braking, and track profiles. Outputs were measured in high temporal resolution to enable detailed comparison. This work provides an indispensable reference point for evaluating simulation fidelity and computational performance. The work further motivates the need for reproducible, scenario-based evaluation systems.

The applications of LTD, especially rapidly evaluated scenarios, is further highlighted in the Wu-Spiryagin-Cole paper. Driving advisory systems are integrated computation engines which advise locomotive operators to take actions based on instrumented vehicle data. These systems require real time evaluation of LTD outputs. Furthermore, energy consumption calculations can be evaluated from a simulator with observable traction/braking forces and speed/distance signals. The accuracy of this energy consumption can be heightened with strong predictions for interactions that are harder to model (handled by the transformer module). Fatigue and safety analyses depend on time-domain histories of dynamic interactions, which can be invaluable for risk assessment.

Table 2.1: Representative longitudinal train dynamics (LTD) simulation tools

Tool	Origin	Scope	Availability
TrainDy [8]	UIC-oriented (academic/industry consortium)	Freight LTD for operations and safety analysis	Specialized / consortium
TOES TM [9, 10]	Developed by the AAR, licensed to member railroads	Comprehensive train dynamics for operating scenarios	Commercial / licensed
TEDS [11, 12]	U.S. FRA-sponsored (contractors)	Safety/risk evaluation, incident investigation, operations LTD	Publicly documented
UM Train [13]	Universal Mechanism suite	Train longitudinal dynamics within multibody ecosystem	Commercial
TDEAS [14]	Developed in China (research/industry)	LTD + energy analysis	Research / institutional
CRE-LTS [7]	Centre for Railway Engineering (CQU)	Benchmarking-ready LTD simulator	Research / in-house

2.2 Data-Driven and Hybrid Modeling in Rail

The computational expense and architectural complexity of high-fidelity rail models have motivated a growing body of work on data-driven and hybrid modeling approaches. These methods utilize machine learning for acceleration of classical simulations, estimation of unmeasured internal quantities, or enabling of real-time decision support. In general, models maintain an interpretable physics-based backbone where possible, but this line is blurred when more advanced "black-box" models are introduced. Existing efforts, in general, fall into three overlapping categories: surrogate replacement of expensive physics submodels, data-driven estimation of in-train forces from operational data, and surrogate modeling for optimization and fleet/network level decision-making.

2.2.1 Surrogate Models for Accelerating Physics-Based Simulation

One prominent line of research focuses on embedding learned surrogate components within classical multibody or finite-element-based simulation frameworks. Instead of replacing the entire physics model with a "black-box" technique, these approaches substitute the most computationally intensive or nonlinear subcomponents with data-driven approximations trained on simulation data. For example, Nie et al. train surrogate elements on finite-element outputs and substitute them for the corresponding mechanical elements inside a multibody vehicle dynamics simulation, coupling MATLAB/Simulink with SIMPACK; they report that a Legendre polynomial regression surrogate substantially reduces computing time without sacrificing accuracy [15]. Polynomial regression, Kriging, and other response-surface techniques are commonly employed in this setting, as they are highly interpretable and allow scientists and engineers to make grounded deductions.

More recent work has emphasized system integration of machine learning surrogates into digital twin architectures. In this schema, a full multibody dynamics (MBD) model is replaced by a neural network based surrogate and packaged using standardized interfaces such as the Functional Mock-up Interface (FMI). This enables deployment as a Functional Mock-up Unit (FMU) within larger co-simulation or decision-support environments [3]. Zhou et al. report that a recurrent neural network surrogate evaluates a 5 km track segment in roughly eight seconds, against approximately thirty minutes for the equivalent multibody simulation in SIMPACK, and they demonstrate a practical pipeline for continuous model deployment. However, this line of work remains primarily focused on reproducing nominal simulator behavior rather than modeling uncertainty or difficult component interactions.

2.2.2 Data-Driven Estimation of In-Train Forces

A second major branch of data-driven rail modeling targets the estimation of in-train forces, which are difficult to measure directly in operational freight and passenger trains. Leveraging the increasing availability of onboard telemetry, particularly from Automatic Train Operation (ATO) systems, several authors have proposed supervised learning approaches that map operational signals such as speed, acceleration, and control commands to coupler force estimates. Convolutional neural networks (CNN) augmented with causal structure and self-attention mechanisms have demonstrated accurate reproduction of simulator-derived force histories under typical operating conditions [16].

Extensions of this concept incorporate multi-task learning and transfer learning to generalize force estimation across multiple couplers and train configurations. The In-trainNet framework, for example, first pre-trains a multi-task model that estimates forces at several couplers of one train configuration, then adapts that pre-trained representation to different configurations through transfer learning, substantially reducing the volume of training data each new configuration requires [17]. While these approaches represent an important step toward scalable “virtual instrumentation” of in-train forces, they are generally trained on labeled data generated by longitudinal train simulators and are validated on limited scenario sets.

2.2.3 Surrogate Modeling for Operations and Optimization

Data-driven surrogates have also been employed to accelerate optimization problems in railway operations, transcending the low-level dynamics and force estimation problems and solving high-level, or even fleet level problems. In the paper from Liu et. al., surrogate models are used to approximate the evaluation of candidate rail schedules, significantly reducing the amount of expensive computation required during optimization [18]. These methods search over multiple surrogates jointly and employ transfer learning, in this case to compensate for incomplete and imbalanced historical disruption data rather than for computational cost alone. Although effective in reducing computational burden, such surrogates are typically designed for specific operational contexts and rely on deterministic predictions of performance metrics for validation.

2.2.4 Limitations of Existing Data-Driven and Hybrid Approaches

Despite demonstrated success on several rail domains, existing data-driven and hybrid rail models exhibit several notable limitations. First, the majority of proposed surrogates are deterministic, providing point estimates without measures of predictive uncertainty. This limits their reliability when deployed outside the training distribution or when embedded within control and optimization loops where constraint violations can have significant consequences.

Second, generalization across operating schemes and consist configurations remains a persistent challenge. Force-estimation models of this kind are typically developed for a single coupler position within a single train configuration, so a separate model must be trained for each coupler and each consist of interest [16]. Subsequent work addresses this limitation directly through multi-task pre-training and transfer learning across configurations, but still requires configuration-specific adaptation data rather than generalizing to an arbitrary consist without retraining [17]. These issues are exacerbated by the reliance on simulated labels, as discrepancies between simulation and real-world behavior can compound errors in purely data-driven mappings.

Finally, while hybrid surrogate approaches retain a physics-based structure, they are generally aimed at replication of nominal simulator outputs rather than modeling of epistemic uncertainty arising from unmodeled dynamics, parameter variability, or environmental disturbances. As a result, current surrogates offer limited robustness under distribution shift, constraining their applicability to safety-critical tasks such as energy-optimal control or real-time operational decision-making. Furthermore, the lack of baseline physical interpretability limits the industry adoption of new simulator technology, as rail engineers and scientists may be skeptical about a fully black-box prediction model (such as neural network).

2.2.5 Implications for Hybrid Probabilistic Modeling

Collectively, this body of work highlights the potential of data-driven methods to accelerate railway simulation and enable new monitoring capabilities, while also revealing fundamental gaps in uncertainty representation and out-of-distribution robustness. These limitations motivate hybrid modeling frameworks that explicitly combine physics-based foundational structure with probabilistic, data-driven components capable of representing uncertainty and adapting to previously unseen operating conditions.

2.3 Transformers for Dynamical Systems and Spatiotemporal Forecasting

Transformer architectures demonstrate strong candidacy for general purpose sequence modeling tasks. They have been increasingly adopted for dynamical systems modeling research and spatiotemporal forecasting. Transformers enable adaptive context aggregation over long temporal horizons by replacing recurrence with multi-head self attention. Unlike recurrent architectures, transformers do not impose a sequential dependency in their core computation. This intrinsic algorithmic feature allows transformers to be highly parallelizable during train-

ing. Transformers are particularly attractive for dynamical systems where future states often depend on long range temporal interactions, as trajectories can be represented as sequences of states or latent tokens [19].

2.3.1 Long-horizon Forecasting Transformers

A major branch of work in transformers is focused on long horizon temporal forecasting. Standard self-attention scales quadratically with sequence length, which severely limits its applicability to long-history prediction problems where fine-temporal resolution is necessary for accuracy. Informer addresses this problem by introducing ProbSparse attention, which identifies and attends to the most informative queries. This enables reduction in the computational cost for training in long horizon scenarios, but preserves enough necessary information to retain prediction accuracy. Compared to conventional models, Informer is capable of forecasting over substantially longer periods [20].

Autoformer is an additional architecture proposed by Wu et. al. which is motivated by the observation that many real-world time series phenomena (such as energy demand, traffic flow, and economic trends) are dominated by two distinct patterns. These include a slowly varying yet long-term trend, and repeating or periodic behavior (seasonal). At great computational cost, a neural network could be used to discover these structures implicitly. Autoformer builds these phenomena directly into the model, where the input time series is first decomposed into trend and seasonal components to enable the model to address long term evolution and short term noise/fluctuations separately. Further, Autoformer replaces scaled dot-product attention with an auto-correlation mechanism that explicitly searches for repeating temporal patterns across various time windows. If recurrent structures are identified, they can be reused more efficiently. Thus, Autoformer improves prediction accuracy and stability for long time series systems by reducing the computational burden of learning every single interaction between time steps [21].

Another promising architecture is the Temporal Fusion Transformer, which are designed for practical forecasting problems where predictions depend on many different types of information, not just a single time series data history. In applications, forecasting requires a combination of static attributes that do not change over time (such as system configuration or asset type), inputs that are known in advance (such as schedules, calendars, or planned control actions), and external variables that are observed over time but not controlled (such as weather or demand signals). Temporal fusion transformers partition these inputs instead of processing them all as a single, aggregate sequence. The model features mechanisms to determine which variables are the most relevant for prediction at each point in time. This structure allows TFT to generate multi-step forecasts while also providing interpretable information about which inputs most strongly influenced the predictions and when they were important, making the model more transparent and suitable for decision-support applications [22]. These properties make the model more transparent and interpretable, as compared to a black-box model.

2.3.2 Neural Operators for PDE-governed Dynamics

For spatiotemporal physical systems, forecasting can be cast as learning a solution operator mapping initial/boundary conditions (and coefficients/forcing) to future fields. Fourier Neural Operators (FNO) address forecasting and dynamical modeling problems from predicting individual timesteps to learning the governing function for how a system evolves. Instead of operating at specific values, FNO learns how information propagates across the entire spatial domain at once. As the name suggests, it does this by transforming the system state into the frequency domain, where long-term or broad patterns can be analyzed and computed much more efficiently. Within this new domain, the model applies global updates that resemble system-wide interactions rather than local point to point updates. As these learned updates are defined independently of any spatial configuration, the resulting model can be evaluated on finer or coarser spatial grids than those seen during training. This allows FNO based models to generalize across different spatial resolutions, making them well suited as fast, flexible surrogates for complex dynamical simulation [23].

Physics-informed neural operators (PINO) expand on operator-learning models by explicitly incorporating known physical laws into the training process. In many engineering or scientific systems, the governing equations for how the system evolves over time are at least partially known. PINO leverages this knowledge by penalizing violations of the underlying physical formulation (the differential equations governing the system) during training, rather than relying solely on error measurements from data. In practice, the model may be trained using available simulation or measurement data at a relatively coarse spatial or temporal resolution, while simultaneously evaluating how well predictions satisfy the governing physical equations at a finer resolution. The resulting model is a generalizable forecaster that respects known physical constraints. The framework allows for the creation of models even when data is limited or expensive to obtain [24].

Beyond representations that rely only on Fourier representations, Cao investigates how attention mechanisms (which were originally created to model sequences) can be used to directly learn the rules of evolution for dynamical systems [25]. The study compares Fourier-based representations, which emphasize global frequency patterns, with Galerkin-style representations, which are rooted in classical numerical methods that approximate solutions using weighted basis functions. In analyzing how self-attention interacts with these different representations, the work shows that no single formulation is universally optimal. The choice depends on the structure of the underlying system and the types of interactions that dominate its dynamic evolution. This perspective reframes operator learning as an architectural design problem, where decisions about how to represent space, enforce structure, and aggregate information are as important as the learning algorithm itself.

More recently, attention-based models have been adapted specifically for learning the evolution of systems governed by partial differential equations. In this line of work, Transformer-style architectures are reformulated to learn solution operators that map initial conditions, system parameters, and external inputs directly to future system states. As opposed to advancing the state forward step by step in time, the model learns a global input–output relationship that captures how the entire state evolves. The attention mechanism enables the model to aggregate information across spatial locations and time, allowing it to represent long-range interactions that are common in physical systems. Utilizing Transformer architec-

tures within an operator-learning framework and evaluating them on canonical PDE benchmarks, this work demonstrates that attention-based models can serve as effective and general surrogates for complex spatiotemporal dynamics, further validating the use of Transformer-style mechanisms in scientific and engineering modeling contexts [26].

2.3.3 Implications for Train Dynamics and Spatiotemporal Decision Support

These elements motivate hybrid modeling strategies for systems like train longitudinal dynamics where dynamics are structured (physics, constraints, conservation-like relationships) yet exhibit long-range dependencies (consist interactions, route profile memory, control plans). Forecasting-oriented Transformers (Informer/Autoformer/TFT) supply scalable mechanisms for long-horizon sequence prediction and covariate fusion [20–22], while neural-operator perspectives (FNO/PINO and attention-based operator learners) suggest a principled route to learning dynamics as operators that map route/consist/control inputs to trajectories or distributions over outcomes [23–26]. These approaches collectively justify treating dynamical response prediction as (i) sequence-to-sequence forecasting with structured attention and (ii) operator learning with physics-aware regularization, providing a foundation for hybrid physics–Transformer surrogates in spatiotemporal control and planning.

2.4 Probabilistic Forecasting and Uncertainty Calibration

In engineered systems, point predictions alone are not sufficient for supporting reliable decision-making. In safety-critical domains, decisions must account for both the expected system behavior as well as the likelihood and severity of rare or adverse outcomes. In such environments, uncertainty is not a nuisance to be minimized, but a fundamental quantity that must be modeled, quantified, and communicated explicitly. Probabilistic forecasting provides a well-studied framework for this purpose by representing predictions as distributions rather than a single value output.

Probabilistic forecasts allow for estimation of quantities relevant to decision making such as prediction intervals, quantiles, and exceedance probabilities. These quantity estimations become directly actionable in downstream tasks such as risk-aware control, constraint satisfaction, and safety margins. For example, rather than predicting a single value representing a failure or violation, a probabilistic model can estimate the probability that a failure will happen.

A central concept of probabilistic forecasting is calibration, and refers to the statistical consistency between predicted probabilities and observed empirical frequencies. A probabilistic model is well-calibrated if events predicted to occur with probability p indeed occur approximately p fraction of the time in practice. Calibration is essential in safety-critical applications, where overconfident predictions can lead to gross underestimation of risk and potentially catastrophic decisions. Poor calibration undermines trust in probabilistic outputs, regardless of the apparent sharpness or sophistication of the predictive model.

However, good calibration alone is not sufficient for a strong probabilistic forecasting model. Trivially wide prediction intervals or overly diffuse distributions may be perfectly calibrated yet provide little useful information for decision-making. This motivates the complementary notion of sharpness, which measures the concentration or informativeness of probabilistic forecasts. Sharpness is a property of the predictive distribution itself, independent of the realized outcomes. Sharpness reflects the model’s ability to make precise predictions. The fundamental tension between calibration and sharpness is well established in the forecasting literature: improving one often degrades the other. Evaluation of probabilistic models must therefore consider both aspects jointly rather than optimizing either in isolation.

In practice, many modern machine learning models produce uncertainty estimates with poor calibration, even when they achieve strong point prediction accuracy. This issue is exacerbated in deep learning models, which are often highly overconfident and sensitive to distributional shift. For safety-critical systems, this behavior is unacceptable, as it may systematically underestimate tail risk. Consequently, conservative uncertainty estimates are often preferred, even at the expense of reduced sharpness, provided that statistical guarantees can be established.

2.4.1 Foundations of Probabilistic Regression

Probabilistic regression aims to characterize the conditional distribution of a target variable given observed inputs, rather than producing a single point estimate. This perspective has a long history in statistics and has gained renewed importance in modern machine learning as uncertainty quantification has become central to decision-making in safety-critical and risk-sensitive applications. Existing approaches to probabilistic regression can be broadly categorized into quantile-based methods, Bayesian formulations, and likelihood-based probabilistic models, each of which encodes uncertainty in a fundamentally different manner.

Quantile regression, introduced by Koenker and Bassett [27], generalizes classical least-squares regression by estimating conditional quantiles of the response variable rather than its conditional mean. By minimizing an asymmetric absolute loss function, often referred to as the pinball or check loss, quantile regression provides a direct and distribution-free mechanism for estimating specific quantiles of the conditional outcome distribution.

This formulation has made quantile regression particularly attractive in applications where tail behavior and asymmetric risk are of interest. Unlike likelihood-based approaches, quantile regression does not require explicit assumptions about the parametric form of the underlying distribution, and quantiles can be estimated independently at arbitrary probability levels. However, classical quantile regression also exhibits important limitations. Independently estimated quantiles are not guaranteed to be mutually consistent, leading to the possibility of quantile crossing, and the method does not provide formal guarantees on coverage. As a result, estimated quantiles may systematically under- or over-cover the true conditional distribution, especially under model misspecification or distributional shift.

Bayesian neural networks represent a fundamentally different approach to probabilistic regression, in which uncertainty is modeled through probability distributions over model parameters rather than through direct estimation of output quantiles. In this framework, predictive uncertainty arises from marginalization over a posterior distribution on network

weights. Variational inference methods have been proposed to make Bayesian neural networks tractable for large-scale models, most notably the Bayes by Backprop approach of Blundell et al. [28], which approximates the posterior distribution over weights using parametric variational families.

Approximate Bayesian techniques such as Monte Carlo dropout further reduce computational complexity by showing that dropout training already corresponds to variational inference in a deep Gaussian process, so that leaving dropout active at test time yields Monte Carlo samples from an approximate predictive posterior [29]. These methods have been widely adopted due to their ease of implementation and compatibility with standard deep learning architectures. Bayesian neural networks and their approximations are often interpreted as capturing epistemic uncertainty arising from limited data or model uncertainty.

Despite their conceptual appeal, Bayesian neural networks do not inherently guarantee calibrated predictive distributions. The quality of uncertainty estimates depends strongly on the choice of prior, the expressiveness of the variational approximation, and the optimization procedure. In practice, approximate inference error and overparameterization can lead to miscalibrated predictions, particularly in regions of the input space that are sparsely represented in the training data.

In addition to quantile-based and Bayesian approaches, many modern regression models produce probabilistic outputs by directly parameterizing a predictive likelihood. Common formulations assume a parametric distribution, such as a Gaussian with learned mean and variance, and train the model using maximum likelihood objectives. More expressive variants, including mixture density networks, extend this approach by modeling multimodal conditional distributions.

Likelihood-based probabilistic regressors offer strong expressive power and are straightforward to integrate into deep learning pipelines. However, their probabilistic validity depends critically on the correctness of the assumed likelihood model. When model assumptions are violated, learned uncertainty estimates may be systematically biased, resulting in overconfident or underconfident predictions. Empirical evidence suggests that such models can achieve high point prediction accuracy while producing poorly calibrated uncertainty estimates, limiting their reliability in downstream decision-making.

The existing literature on probabilistic regression provides a diverse set of tools for uncertainty-aware prediction. Quantile regression offers a simple and distribution-free approach to estimating conditional quantiles but lacks formal coverage guarantees. Bayesian neural networks provide a principled framework for modeling epistemic uncertainty but are sensitive to approximation quality and modeling assumptions. Likelihood-based probabilistic regressors are flexible and scalable but often exhibit miscalibrated uncertainty in practice.

These limitations motivate subsequent work on uncertainty calibration and distribution-free guarantees. In particular, methods that explicitly evaluate and correct calibration, as well as techniques that provide finite-sample coverage guarantees independent of model correctness, have emerged as important complements to expressive probabilistic regression models.

2.4.2 Empirical Miscalibration in Deep Models

Despite their expressive power and strong performance in point prediction tasks, modern deep learning models frequently produce probabilistic predictions with poor calibration. This phenomenon has been systematically documented by Guo et al. [30], as they demonstrated that contemporary neural networks tend to produce overconfident results, even when trained with likelihood-based objectives and regularization techniques. In both classification and regression settings, predicted probabilities often fail to correspond to empirical frequencies. This damages trust in the model to produce reliable estimates in downstream decision-making.

A major contribution of this thread of work is the formalization and adoption of calibration diagnostics for neural networks. Metrics such as expected calibration error (ECE) and reliability diagrams provide both quantitative and visual tools to analyze whether predicted confidence levels align with observed outcomes. Empirical results indicate that increasing model capacity, depth, and training duration can contribute to miscalibration, despite improvements in predictive accuracy. These findings challenge the assumption that probabilistic outputs produced by deep models are inherently reliable.

Miscalibration issues are not confined to only deterministic neural networks. Bayesian neural networks and approximate Bayesian methods (such as variational inference and Monte Carlo dropout) have also been shown to produce miscalibrated predictive distributions. Approximation error in posterior inference, sensitivity to prior specification, and artifacts of optimization can all contribute to overconfident uncertainty estimates. As a result, Bayesian formulations alone do not guarantee calibrated predictions, particularly in high-dimensional or data-sparse schemes.

The presence of miscalibration in deep probabilistic models has significant implications for safety-critical applications. Overconfident predictions can lead to systematic underestimation of risk, while underconfident predictions may result in overly conservative behavior and reduced efficiency. These observations motivate the need for uncertainty quantification methods that go beyond expressive modeling and explicitly address calibration, evaluation, and robustness. Subsequent work has therefore focused on calibration-aware learning objectives, distribution-free guarantees, and evaluation frameworks that jointly consider calibration and sharpness, which are reviewed in the following sections.

2.4.3 Post-hoc Calibration for Regression

The empirical miscalibration observed in modern probabilistic models (especially deep models) has motivated the development of post-hoc calibration methods that operate independently of the underlying predictive model. Kuleshov et al. [31] formalized calibration for regression by defining a notion of distributional consistency based on predictive cumulative distribution functions. Under this framework, a probabilistic regressor is calibrated if the empirical frequency of outcomes falling below predicted quantiles matches the nominal quantile levels.

Expanding on this definition, Kuleshov et al. proposed a general recalibration procedure that maps predicted cumulative probabilities to calibrated probabilities using held-out validation data. This approach is model-agnostic and can be applied to a wide range of

probabilistic regressors, including Bayesian neural networks and likelihood-based deep models. Empirical results demonstrate substantial improvements in calibration with minimal degradation in predictive sharpness, making calibrated regression a practical and widely applicable correction mechanism.

However, post-hoc calibration methods rely on the assumption that calibration error is stable across training and deployment distributions. While effective in correcting average miscalibration on validation data, these techniques do not provide formal guarantees on predictive coverage and may fail under distributional shift or in sparsely sampled regions of the input space. As a result, post-hoc calibration improves empirical reliability but does not eliminate the need for uncertainty estimation methods with stronger statistical guarantees.

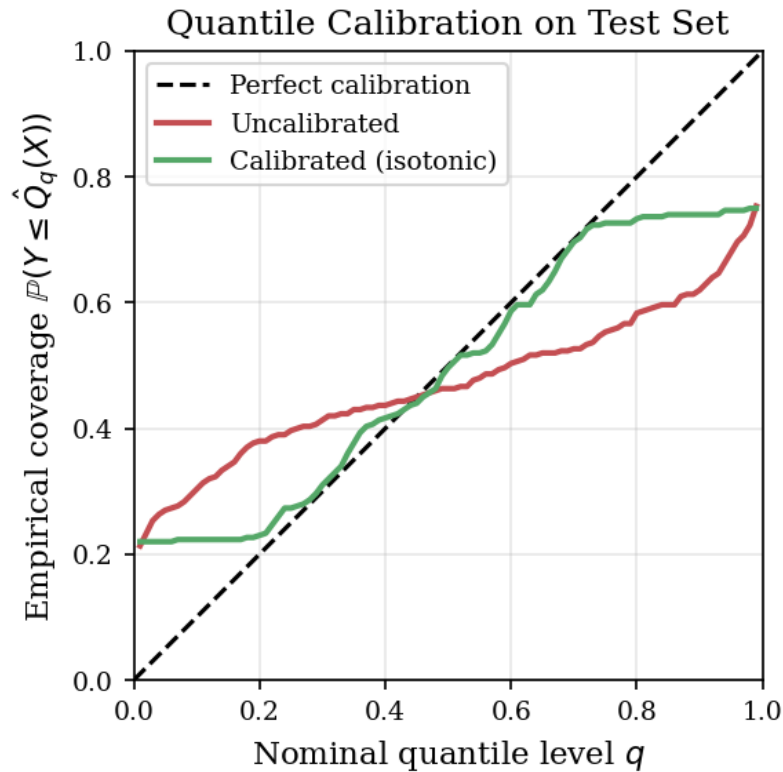


Figure 2.1: Quantile calibration: diagonal = perfect; uncalibrated model deviates; isotonic calibration corrects.

Figure 2.1 illustrates the distinction. A well-calibrated model produces quantiles such that when we predict the q -quantile $\hat{Q}_q(X)$, the fraction of held-out outcomes $Y \leq \hat{Q}_q(X)$ is approximately q . The diagonal represents this ideal. An uncalibrated model may deviate: for example, an overconfident model (underestimated variance) yields empirical coverage below the diagonal at high nominal levels. Post-hoc calibration methods such as isotonic regression can correct this by learning a monotone mapping from predicted to calibrated quantile levels on a validation set.

2.4.4 Distribution-Free Coverage via Conformal Prediction

Conformal prediction offers an alternative approach to uncertainty quantification by providing finite-sample, distribution-free coverage guarantees under minimal assumptions. Introduced by Shafer and Vovk [32], conformal prediction constructs prediction sets that contain the true outcome with a specified probability, provided the data are exchangeable. Crucially, these guarantees hold regardless of the correctness of the underlying predictive model.

Recent work has integrated conformal prediction with quantile regression to produce calibrated prediction intervals that adapt to heteroscedasticity in the data. Romano et al. [33] proposed conformalized quantile regression, which adjusts estimated conditional quantiles to ensure exact marginal coverage. This method preserves the flexibility of quantile regression while enforcing rigorous statistical guarantees, making it particularly attractive for safety-critical applications.

The primary limitation of conformal prediction methods is their inherent conservatism. Coverage guarantees are typically marginal rather than conditional, and the resulting prediction intervals may be wider than necessary for individual inputs. This trade-off between coverage guarantees and predictive sharpness reflects a fundamental tension in uncertainty quantification. Regardless, conformal prediction provides a robust and model-agnostic mechanism for uncertainty estimation, complementing calibration-based approaches and motivating hybrid methods that balance expressiveness, calibration, and statistical rigor.

2.4.5 Proper Scoring Rules and Evaluation Metrics

Evaluating probabilistic forecasts requires metrics that jointly assess both calibration and sharpness. Proper scoring rules provide a principled framework for this purpose by assigning a numerical score to a predicted distribution such that the expected score is minimized when the predictive distribution matches the true data-generating distribution. Gneiting and Raftery [34] formalized this framework and emphasized that calibration and sharpness must be considered together rather than as competing objectives.

Commonly used proper scoring rules include the logarithmic score, the continuous ranked probability score (CRPS), and interval-based scores. Unlike point-error metrics, proper scoring rules penalize both overconfident and overly diffuse predictions, making them well suited for comparing probabilistic models. As a result, they have become standard tools for evaluating probabilistic forecasts across a wide range of applications, including time series prediction and uncertainty-aware regression.

In the context of safety-critical forecasting, proper scoring rules provide an objective means of assessing whether probabilistic predictions are both reliable and informative. Their use complements calibration diagnostics and coverage-based guarantees, offering a unified evaluation framework for probabilistic regression models.

Synthesis and Implications

Taken together, the literature reviewed in this chapter highlights both the maturity and the limitations of existing approaches to train dynamics modeling and decision support. Classical

longitudinal dynamics models provide well-understood physical structure and interpretability but remain computationally expensive and difficult to integrate into optimization and control pipelines. Data-driven and hybrid modeling efforts have demonstrated the feasibility of surrogate models for accelerating simulation, estimating in-train forces, and supporting operational decision-making, yet these approaches are typically deterministic, limited in their ability to generalize outside the training distribution, and rarely equipped with rigorous uncertainty quantification. Recent advances in transformer architectures for dynamical systems and spatiotemporal forecasting offer powerful tools for modeling long-horizon dependencies and high-dimensional temporal structure, but their application to rail systems remains largely unexplored and their uncertainty estimates are often informal or empirically calibrated at best. Finally, the probabilistic forecasting literature establishes that expressive models alone do not yield reliable uncertainty estimates, motivating the use of calibration techniques, distribution-free coverage guarantees, and proper scoring rules to ensure statistical reliability in safety-critical settings. Collectively, these findings suggest a gap between physically grounded simulation, expressive data-driven models, and trustworthy probabilistic outputs. This gap motivates the development of hybrid modeling frameworks that combine physics-based structure, transformer-style sequence modeling, and explicitly calibrated probabilistic forecasting to support risk-aware train dynamics simulation and decision-making.

Chapter 3

Problem Formulation

3.1 The prediction and control problem, and the tensor format used to describe it

This chapter defines the problem this thesis solves and the mathematical objects used to describe it. The physical picture is longitudinal train dynamics (LTD): a chain of vehicles connected by couplers, moving along a route, pushed and slowed by traction, braking, resistance, grade, and the forces passed through the couplers themselves. None of this physics is new. What is new is how it is packaged. This thesis represents the coupled-vehicle system as a small set of tensors, structured arrays, so that the simulator’s outputs, the model’s training targets, and the surrogate model built in later chapters all share the same layout, no matter how many vehicles are in the train.

The rest of this chapter proceeds as follows. Section 3.1.1 states, in plain terms, what is being predicted and why it matters. Section 3.1.2 restates the underlying physics in its usual form. Sections 3.1.3 and 3.1.4 introduce the node and edge tensors and rewrite the dynamics in that format. Section 3.1.5 states the learning objective, including the requirement that the learned model be differentiable so that it can later be used to choose control actions, not just predict outcomes. The reference simulator that generates training data from this formulation is documented separately, in Chapter 4.

3.1.1 Physical setting and prediction task

Picture a freight train moving along a route. Before it runs, or while planning how it should run, a few questions matter: how much fuel will this trip use, will it arrive on time, and will the forces between cars ever get dangerously high. A more useful question is how the answers to those change if the driving plan is adjusted slightly.

Answering that last question well is the real goal of this thesis. A model that can only report what happened in one specific run is a forecasting tool. A model that can also say how the outcome would shift if traction or braking commands were changed, and can do so cheaply and repeatedly, becomes something a control algorithm can use to choose better driving plans on its own. That is the target: a model fast and smooth enough to sit inside an optimization loop that picks fuel-efficient throttle and brake commands, while staying

within speed limits and coupler-force safety limits.

To make this concrete, every scenario is described by three things: a route profile (the track's grade, curvature, and speed limits along its length), a consist configuration (the ordered list of vehicles, their masses, resistance properties, and coupler characteristics), and a control or operating plan (the traction and braking commands applied over time, possibly adjusted by a driver or automation model). Given these three inputs, the quantities of interest include:

- **Motion:** the position, velocity, and acceleration of every vehicle.
- **In-train forces:** the force at each coupler, which determines whether neighboring cars are pulling apart (draft) or pushing together (buff).
- **Energy and fuel:** the work or power used over the run.
- **Schedule:** trip time and arrival at scheduled points.
- **Safety margin:** how close any of the above comes to an operating limit.

This thesis treats the physics simulator described in Chapter 4 as ground truth when its parameters are fixed. Uncertainty in those parameters, or in the environment, is folded in as well (section 3.4), but its role here is narrower than a full probabilistic forecast: it mainly shows up as a safety margin that the control algorithm has to respect, rather than as a result reported for its own sake.

3.1.2 Classical longitudinal dynamics formulation

Classical longitudinal modeling represents the train as a chain of N vehicles indexed $i = 0, \dots, N - 1$ from lead to tail, connected by $N - 1$ couplers. Let $x_i(t)$ and $v_i(t) = \dot{x}_i$ denote the longitudinal position and velocity of vehicle i along the route, and let m_i denote its mass. Between vehicles i and $i + 1$, coupler i is associated with a kinematic extension δ_i (relative displacement from nominal spacing, positive in draft in the convention used below) and a coupler force F_i transmitted to the trailing vehicle in the pair. Resistance, grade, and optional curvature terms enter as external longitudinal forces on each vehicle; traction and braking appear as controlled inputs, possibly after actuator dynamics. Summarizing the external and control contributions on vehicle i by $F_{\text{ext},i}(t)$ and the coupler incidence by the force on the rear of i minus the force on the front, the translational dynamics take the familiar form

$$\dot{x}_i = v_i, \quad m_i \dot{v}_i = F_{\text{ext},i}(t) + F_{i-1}(t) - F_i(t), \quad (3.1)$$

with $F_{-1} = F_{N-1} = 0$ when end forces are absent. The coupler forces F_i are determined by constitutive laws $F_i = \phi_i(\delta_i, \dot{\delta}_i, \dots)$ linking kinematics to internal forces; slack, asymmetric buff and draft stiffness, and damping enter here (chapter 4). When actuator lag or power limits are modeled, eq. (3.1) is augmented by additional state variables per vehicle (for example brake and traction command states) driven by exogenous commands. This is the standard description of a coupled multi-body train, and it holds regardless of the tensor notation introduced next.

3.1.3 Tensorized state representation

For learning, and for software that must handle variable train length and rich per-vehicle features, it is convenient to organize the same physics into two matrices that make the vehicle-coupler graph explicit. In this thesis, a *node* (vehicle) tensor $\mathbf{H}(t) \in \mathbb{R}^{N \times d_n}$ collects d_n channels associated with each vehicle at time t . Typical channels include longitudinal position and velocity, internal actuator or command states when those are part of the dynamic model, and static attributes replicated on each row (mass, Davis coefficients, traction capability flags, force limits, and similar parameters) so that a single array suffices for batched computation and for models that read local vehicle features. An *edge* (coupler) tensor $\mathbf{E}(t) \in \mathbb{R}^{(N-1) \times d_e}$ holds d_e channels per coupler, such as extension δ_i , relative speed $\dot{\delta}_i$, algebraic coupler force F_i , nominal spacing, slack half-width, and stiffness and damping parameters for buff and draft segments. The assignment of channels is fixed in the reference implementation; the important point is the separation between vehicle-centered quantities in $\mathbf{H}$ and coupler-centered quantities in $\mathbf{E}$.

Along a discrete time grid $t_0, \dots, t_{T-1}$, simulated trajectories can be stored as a *rollout tensor* $\mathbf{X} \in \mathbb{R}^{T \times N \times d}$, whose time slice $\mathbf{X}_{k,:,:}$ is the node feature matrix at t_k (so one may take $d = d_n$ and identify $\mathbf{X}_{k,:,:}$ with $\mathbf{H}(t_k)$ after any bookkeeping for channels). Coupler histories are stored analogously as $\mathbf{E}_{\text{hist}} \in \mathbb{R}^{T \times (N-1) \times d_e}$.

This layout keeps the structure of the physical problem visible: each coupler row depends only on the two vehicle rows next to it, mirroring the chain shape of eq. (3.1). That chain shape is also why this thesis treats the train as a graph, with vehicles as nodes and couplers as edges, rather than as a fixed-length sequence. A graph-based model can process trains of any length directly, passing information between neighboring vehicles one coupler at a time, without needing to pad shorter trains or mask out unused positions the way sequence models do. Handling trains of different lengths this way, as a direct consequence of the representation rather than as an engineering workaround, is central to the train-agnostic goal of this thesis: the same trained model should work on a train it has never seen, of a length it has never seen, without modification.

3.1.4 Dynamics in tensor-oriented form

The tensor view is a reorganization of state and parameters, not a change to the underlying force balances. Let $\mathbf{H}_{\text{dyn}}(t)$ denote the dynamic channels within $\mathbf{H}(t)$ (for example positions, velocities, and actuator states in the extended reference model), and let $\mathbf{p}$ collect fixed parameters, including static columns of $\mathbf{H}$ and of $\mathbf{E}$ where those encode coupler constants. The differential part of the evolution can be written compactly as

$$\frac{d}{dt} \text{vec}(\mathbf{H}_{\text{dyn}}(t)) = \mathcal{F}_n(\mathbf{H}(t), \mathbf{E}(t), \mathcal{U}(t), \mathcal{R}; t), \quad (3.2)$$

where $\mathcal{U}$ denotes the control input signals, $\mathcal{R}$ the route profile, and $\mathcal{F}_n$ the assembled right-hand side obtained from eq. (3.1) and the actuator relations. Coupler kinematics link $\mathbf{H}_{\text{dyn}}$ to the extension and rate channels in $\mathbf{E}(t)$; constitutive laws determine the force channels. In the reference integrator used for data generation in this thesis, several edge channels (including δ_i , $\dot{\delta}_i$, and F_i) are *reconstructed algebraically* at each evaluation from vehicle positions and

velocities and from coupler parameters, rather than integrated as separate states. Thus the edge tensor is in part an explicit bookkeeping of quantities that satisfy relations of the form

$$\mathbf{E}(t) = \mathcal{G}_e(\mathbf{H}(t), \mathbf{p}), \quad (3.3)$$

for the algebraic channels, while eq. (3.2) carries the true differential degrees of freedom. If additional internal coupler or draft-gear states were introduced in future work, eq. (3.3) would be supplemented by differential equations for the corresponding rows of $\mathbf{E}(t)$, i.e. a nontrivial $\mathcal{F}_e$ on those components alongside $\mathcal{F}_n$.

The surrogate model built later in this thesis does not try to learn $\mathcal{F}_n$ from a blank slate. Instead, it starts from this known physics right-hand side and learns a correction on top of it, so that the predicted rate of change takes the form $\mathcal{F}_n + \Delta\mathcal{F}_\theta$, where $\Delta\mathcal{F}_\theta$ is a small learned term produced by a graph neural network. Building the physics into the model this way, rather than treating it as just another input feature or as a soft penalty added to the loss, gives the surrogate a strong head start and keeps its behavior anchored to something physically sensible even for consists and routes it has not seen during training.

3.1.5 Forecasting objective and simulator operator

The learning problem is to predict, under computation and uncertainty constraints, the outputs described in section 3.1.1 from the structured inputs $(\mathcal{R}, \mathcal{C}, \mathcal{U})$. The reference physics implements a deterministic operator

$$F : (\mathcal{R}, \mathcal{C}, \mathcal{U}) \mapsto \mathbf{y}, \quad (3.4)$$

where $\mathbf{y}$ collects trajectories (for example stacks of $\mathbf{H}(t_k)$ and $\mathbf{E}(t_k)$), scalar summaries such as energy and trip time, and any derived risk indicators. The triple $(\mathcal{R}, \mathcal{C}, \mathcal{U})$ denotes the same structured inputs as the route profile $\mathcal{R}$, consist $\mathcal{C}$, and control plan $\mathcal{U}$ detailed in section 3.2. When uncertain parameters and environment are modeled by a random vector $\xi \sim p(\xi)$, the same operator is written $F(\mathcal{R}, \mathcal{C}, \mathcal{U}; \xi)$ and outputs become random (section 3.4).

The goal of this thesis is not just to match individual trajectories point by point. The surrogate needs to be accurate enough that its outputs, especially coupler forces and trip energy, can be trusted for planning and for staying within safety margins. Where uncertainty matters, it mainly shows up as a margin that the surrogate and the downstream control algorithm must respect, rather than as a full calibrated probability distribution reported over every output.

The surrogate introduced later in this thesis approximates F using a message-passing graph neural network that operates directly on the node and edge tensors defined above, learning a correction to the known physics rather than replacing it (section 3.1.4). Because the surrogate must eventually be used to choose control actions, not just predict outcomes for a fixed plan, it is also built to be differentiable: it can be evaluated many times in a batch, and the gradient of its outputs with respect to the control plan can be computed directly. That property is what later makes it possible to use the surrogate as the model inside a fuel-optimal control routine, instead of only as a fast forecasting tool.

3.2 Input definitions

3.2.1 Route profile

A route profile $\mathcal{R}$ provides the exogenous fields sampled by each vehicle as it moves along the track. In the reference simulator, $\mathcal{R}$ is a piecewise representation over distance s that supports interpolation of grade, curvature, and speed constraints:

- grade field $\theta(s)$ (or $\sin \theta(s)$ in implementation),
- curvature proxy $\kappa(s)$ when enabled,
- operational constraints such as speed limits $v_{\max}(s)$.

Given vehicle positions $x_i(t)$ from the node tensor, route-dependent forces are evaluated per vehicle at $s = x_i(t)$. This keeps the forcing consistent with the tensorized state layout while preserving the classical physics of section 3.1.2.

Route profiles used in this thesis are built from real elevation and track alignment data rather than invented from scratch, so that the grades and curves seen during training and evaluation resemble those of an actual rail corridor. Extra scenarios are produced by mildly perturbing these real profiles, for example in grade severity or adhesion, rather than by generating routes with no real-world basis. The pipeline that turns raw map and elevation data into these profiles is documented in chapter 5.

3.2.2 Consist configuration

A consist configuration $\mathcal{C}$ defines the ordered vehicle chain and coupler parameters. It is represented as two aligned tables that map directly to tensor channels:

- **Vehicle attributes** for $i = 0, \dots, N - 1$: mass, Davis coefficients, traction/brake limits, and actuator constants.
- **Coupler attributes** for $j = 0, \dots, N - 2$: nominal spacing, slack half-width, buff/draft stiffness, and damping parameters.

Static vehicle attributes are stored as fixed channels of the node tensor $\mathbf{H}$, and static coupler attributes as fixed channels of the edge tensor $\mathbf{E}$. This organization is exactly the data schema used by the simulator and by the surrogate model.

3.2.3 Control plan

A control plan $\mathcal{U}$ specifies exogenous command signals over time (or equivalent distance-indexed schedules mapped to time during simulation). For each powered or braked vehicle, commands are represented as channels $u_{\text{trac},i}(t)$ and $u_{\text{brk},i}(t)$ driving the actuator dynamics documented in chapter 4. The simulator therefore receives structured inputs as the triplet $(\mathcal{R}, \mathcal{C}, \mathcal{U})$ introduced in eq. (3.4), and produces tensorized trajectories and derived summaries.

3.3 Output definitions

The output of one simulation run is a structured object

$$\mathbf{y} = (\mathbf{X}, \mathbf{E}_{\text{hist}}, \mathbf{s}, \mathbf{r}), \quad (3.5)$$

where $\mathbf{X}$ and $\mathbf{E}_{\text{hist}}$ are rollout tensors, $\mathbf{s}$ contains scalar summaries, and $\mathbf{r}$ contains risk-oriented statistics.

3.3.1 Trajectory-level outputs

At discrete times t_k , the node rollout is $\mathbf{X}_{k,:} \in \mathbb{R}^{N \times d_n}$ and the edge rollout is $\mathbf{E}_{\text{hist},k,:} \in \mathbb{R}^{(N-1) \times d_e}$. These tensors contain the channels defined in section 3.1.3: per-vehicle motion and actuator states in $\mathbf{X}$, and per-coupler extension, relative speed, and force channels in $\mathbf{E}_{\text{hist}}$. Classical quantities are recovered directly from selected channels, e.g. $x_i(t_k)$, $v_i(t_k)$, and coupler force $F_j(t_k)$. For notation, the coupler force channel may be written as

$$F_j(t) = f_j(\mathcal{R}, \mathcal{C}, \mathcal{U}, t), \quad (3.6)$$

where f_j is induced by the simulator dynamics and constitutive laws.

3.3.2 Aggregate outputs

Aggregate outputs are deterministic functionals of rollout tensors for fixed parameters. Typical summaries used throughout this thesis include

$$E_{\text{trip}} = \int_0^T P_{\text{trac}}(t) dt, \quad (3.7)$$

$$T_{\text{arr}} = \inf\{t : x_{\text{lead}}(t) \geq s_{\text{target}}\}, \quad (3.8)$$

$$F_{\text{max}} = \max_{t,j} |F_j(t)|. \quad (3.9)$$

All such metrics are post-processing maps on $(\mathbf{X}, \mathbf{E}_{\text{hist}})$ and therefore remain consistent with the tensorized representation.

3.3.3 Risk metrics

For uncertainty-aware forecasting, risk is expressed as probabilities and quantiles of tensor-derived observables. For threshold τ and quantity Z (for example F_{max} or $|F_j(t)|$), the exceedance probability is

$$\mathbb{P}(Z > \tau). \quad (3.10)$$

Likewise, predictive quantiles satisfy $\mathbb{P}(Y \leq Q_p) = p$ for selected levels p . Both quantities are evaluated on variables computed from node/edge rollouts, so no separate non-tensor output definition is required. In this thesis these risk metrics are used mainly for coupler-force safety, as discussed next.

3.4 Uncertainty sources

Not every input to the simulator is known exactly. Vehicle masses, resistance coefficients, coupler stiffness, the exact shape of the route, and even how precisely a command is executed all carry some uncertainty. This section lists where that uncertainty comes from and how this thesis handles it. Because the main use of the surrogate in this thesis is fuel-optimal control, not general-purpose forecasting, uncertainty is treated mainly as something that narrows the safe operating margin the control algorithm must respect, rather than as a full predictive distribution reported for its own sake.

Uncertainty is introduced through a random vector ξ that perturbs physically meaningful components of the structured input and model:

1. **Vehicle and coupler parameters:** mass, resistance coefficients, slack and stiffness values, brake/traction effectiveness.
2. **Route and environment:** grade/curvature profile errors, adhesion-related effects, and external disturbances.
3. **Operational realization:** command timing and execution variability.
4. **Model discrepancy:** residual mismatch between the reference simulator and real operations.

3.4.1 Formal treatment of uncertainty

Let $\xi \sim p(\xi)$ denote the joint uncertainty law. Then the simulator operator from eq. (3.4) becomes

$$\mathbf{y}(\xi) = F(\mathcal{R}, \mathcal{C}, \mathcal{U}; \xi), \quad (3.11)$$

and every tensor channel and derived metric becomes a random variable. For example, the exceedance probability for coupler interface j is

$$\mathbb{P}_\xi(F_j(t; \xi) > \tau) = \int_{\{\xi: F_j(t; \xi) > \tau\}} p(\xi) \, d\xi. \quad (3.12)$$

In practice, these probabilities are approximated by Monte Carlo simulation or by probabilistic surrogates trained on ensembles generated from the same tensorized simulator. The role this plays in the thesis is narrow: $\mathbb{P}_\xi(F_j > \tau)$ is used to define a safety margin that the fuel-optimal control routine is required to respect, for example by keeping the estimated coupler force below its limit with high probability, rather than as a general-purpose calibrated forecast reported across every output variable.

3.5 Supervised Learning Objective

Training data are generated by the reference simulator documented in chapter 4:

$$\mathcal{D} = \left\{ (\mathcal{R}^{(k)}, \mathcal{C}^{(k)}, \mathcal{U}^{(k)}, \mathbf{X}^{(k)}, \mathbf{E}_{\text{hist}}^{(k)}, \mathbf{s}^{(k)}) \right\}_{k=1}^M. \quad (3.13)$$

The surrogate f_θ maps structured inputs to trajectory and summary predictions in the same tensor layout, predicting a correction on top of the known physics right-hand side (section 3.1.4) rather than the dynamics outright. A generic training objective is

$$\mathcal{L}(\theta) = \lambda_X \mathcal{L}_X + \lambda_E \mathcal{L}_E + \lambda_S \mathcal{L}_S + \lambda_Q \mathcal{L}_Q + \lambda_P \mathcal{L}_P, \quad (3.14)$$

with rollout losses on node and edge tensors, summary losses, a quantile loss for the safety-relevant margin, and an optional physics-consistency penalty.

3.5.1 Loss term definitions

Rollout losses. $\mathcal{L}_X$ and $\mathcal{L}_E$ compare predicted and reference tensor channels over time, vehicle/coupler index, and feature channels (with masking for variable train length where needed).

Summary loss. $\mathcal{L}_S$ penalizes errors on scalar outputs such as trip energy, arrival time, or peak coupler force.

Quantile loss. For target y and predicted quantile $\hat{q}_p$, the pinball loss is

$$\rho_p(y, \hat{q}_p) = \begin{cases} p(y - \hat{q}_p), & y \geq \hat{q}_p, \\ (1 - p)(\hat{q}_p - y), & y < \hat{q}_p, \end{cases} \quad (3.15)$$

and $\mathcal{L}_Q$ aggregates ρ_p over selected quantile levels. In this thesis it is applied narrowly, to the handful of safety-relevant quantities, chiefly coupler force, consistent with the narrower role of uncertainty described in section 3.4, rather than to every predicted channel.

Physics-consistency penalty. $\mathcal{L}_P$ is a small, optional penalty on top of the residual architecture described in section 3.1.4. Because the surrogate already predicts a correction to a known physics right-hand side rather than the full dynamics from scratch, most of the physical plausibility is built into the model's structure and does not need to be enforced after the fact. $\mathcal{L}_P$ only needs to catch smaller inconsistencies, for example a coupler-force channel that does not agree with the accelerations of the two vehicles it connects.

Remark. A later chapter instantiates these terms for the specific graph neural network architecture and training regime used in this thesis.

3.6 Conceptual pipeline

The full workflow, from generating data to using the trained model for control, follows the same tensor objects at every step:

1. **Structured inputs:** Specify $(\mathcal{R}, \mathcal{C}, \mathcal{U})$ and, for uncertainty studies, sample ξ .
2. **Reference simulation:** Run the tensorized LTD simulator (chapter 4) to obtain $(\mathbf{X}, \mathbf{E}_{\text{hist}}, \mathbf{s})$.
3. **Dataset assembly:** Store many such runs as supervised pairs for training and validation.

4. **Surrogate training:** Fit f_θ to predict rollout tensors and derived summaries, including the coupler-force safety margin.
5. **Forecasting:** Use f_θ for fast scenario evaluation in the same output space as the simulator.
6. **Control:** Use f_θ and its gradients inside an optimization routine, such as model-predictive control, to choose traction and braking commands that reduce fuel use while respecting speed limits and coupler-force safety margins.

Figure 3.1: Conceptual pipeline from structured inputs to tensorized simulator outputs, surrogate forecasting, and fuel-optimal control.

3.7 Evaluation targets

Evaluation is defined on the same outputs as the training objective:

Trajectory fidelity. How closely predicted vehicle motion and coupler forces track the reference simulator, channel by channel (for example MAE/RMSE on velocity and coupler-force channels).

Summary accuracy. Errors on trip-level numbers such as total energy used, arrival time, and peak coupler force.

Safety-margin quality. How well the surrogate’s coupler-force margin estimates hold up, since this is what the control routine relies on to stay within limits.

Generalization across trains and routes. Accuracy on consist lengths and route corridors not seen during training. Since the surrogate is meant to work on trains it was never explicitly trained on, this is not a side check; it is one of the central claims of the thesis.

Operational utility. Inference-time speedup relative to the reference simulator, and performance as the model inside the fuel-optimal control routine, compared against a simple constant-speed driving plan.

3.8 Chapter summary

This chapter defined the prediction and control problem this thesis addresses. Classical longitudinal dynamics provides the physical foundation (eq. (3.1)). This thesis recasts that same system into node and edge tensors $\mathbf{H}(t)$ and $\mathbf{E}(t)$ (sections 3.1.3 and 3.1.4), a graph-shaped representation that lets a single model handle trains of any length. The simulator F maps structured inputs $(\mathcal{R}, \mathcal{C}, \mathcal{U})$ to structured outputs $\mathbf{y}$: trajectory tensors, trip-level summaries such as energy and arrival time, and safety-relevant quantities such as peak coupler force (eq. (3.4)). We distinguished deterministic outputs under fixed parameters from outputs under random parameter uncertainty ξ (eq. (3.11)).

The learning task is hard for four reasons: the inputs are high-dimensional and structured, the outputs span different scales and types, physical consistency has to be preserved, and safety-relevant quantities need a notion of margin rather than a single number. This thesis addresses the last of these narrowly, folding uncertainty into a coupler-force safety margin rather than building a full calibrated forecast of every output (section 3.4).

The next chapter documents the reference simulator that generates the data used throughout the rest of this thesis. The chapter after that introduces the surrogate model itself: a graph neural network that learns a correction to the known physics and is trained to be differentiable, so that it can later serve as the model inside a fuel-optimal control routine.

Chapter 4

Reference Simulator

Chapter 3 laid out the prediction and control problem and the tensor format used to represent a train. This chapter builds the tool that actually produces the numbers: a reference simulator of longitudinal train dynamics. Its job is simple to state even though the physics inside it is not. Given a train made of coupled vehicles, a route, and a set of driving commands, the simulator plays out what happens over time and records it. Every training example used later to teach the surrogate model comes from running this simulator once, so in that sense this chapter describes the data generator for the rest of the thesis.

The simulator was built up gradually, one physical effect at a time, so that each addition could be checked before the next was layered on. It follows a Jupyter notebook that constructs the model stage by stage using only NumPy, SciPy’s `solve_ivp`, and Matplotlib, and the same model is mirrored in a structured Python package that exposes the node and edge tensors from chapter 3 alongside the classical flat state used for integration.

The parameters used, vehicle mass, resistance coefficients, coupler stiffness, traction limits, are realistic in shape but are not tuned to match any specific real locomotive or railcar. The goal at this stage is a simulator whose structure is right, not one whose numbers are certified against a real test train. This distinction matters when interpreting later results: the surrogate and the control experiments built on top of this simulator are validated against the simulator itself, not against real-world rail data. Time integration uses adaptive Runge-Kutta or implicit methods as needed when stiff coupler dynamics appear, and the figures in this chapter are exported directly from the notebook at representative numerical settings.

4.1 Stage 1: single powered vehicle

The starting point is a single point mass m with longitudinal position $x(t)$ and velocity $v(t) = \dot{x}$. The equation of motion aggregates traction, braking, Davis running resistance, grade, and an optional curvature proxy into a single scalar force balance:

$$m\dot{v} = F_{\text{trac}}(t) - F_{\text{brk}}(t) - R(v) - G(x) - C(x, v), \quad (4.1)$$

with $\dot{x} = v$. Davis resistance is implemented in the usual magnitude-direction form,

$$R(v) = (A + B|v| + Cv^2) \operatorname{sgn}(v), \quad (4.2)$$

with $\text{sgn}(v)$ set to zero at $|v| < \varepsilon$ to avoid chatter at standstill. Grade enters through $G(x) = mg \sin \theta(x)$ with θ the local track inclination. The optional term C is a simple longitudinal curvature proxy proportional to $mv^2|\kappa(x)|$ with tunable scale, disabled in the earliest runs. Traction and brake commands may be specified as arbitrary functions of time; the first stage uses a smooth “ramp” in time so that F_{trac} rises continuously from zero after a delay, rather than stepping instantaneously, which eases numerical stiffness and mirrors the later actuator-lag extension. Figure 4.1 shows position, speed, and the plotted residual $F_{\text{trac}} - R(v)$ on level track with braking and curvature removed, confirming that the integrator and force signs behave as intended before couplers are introduced.

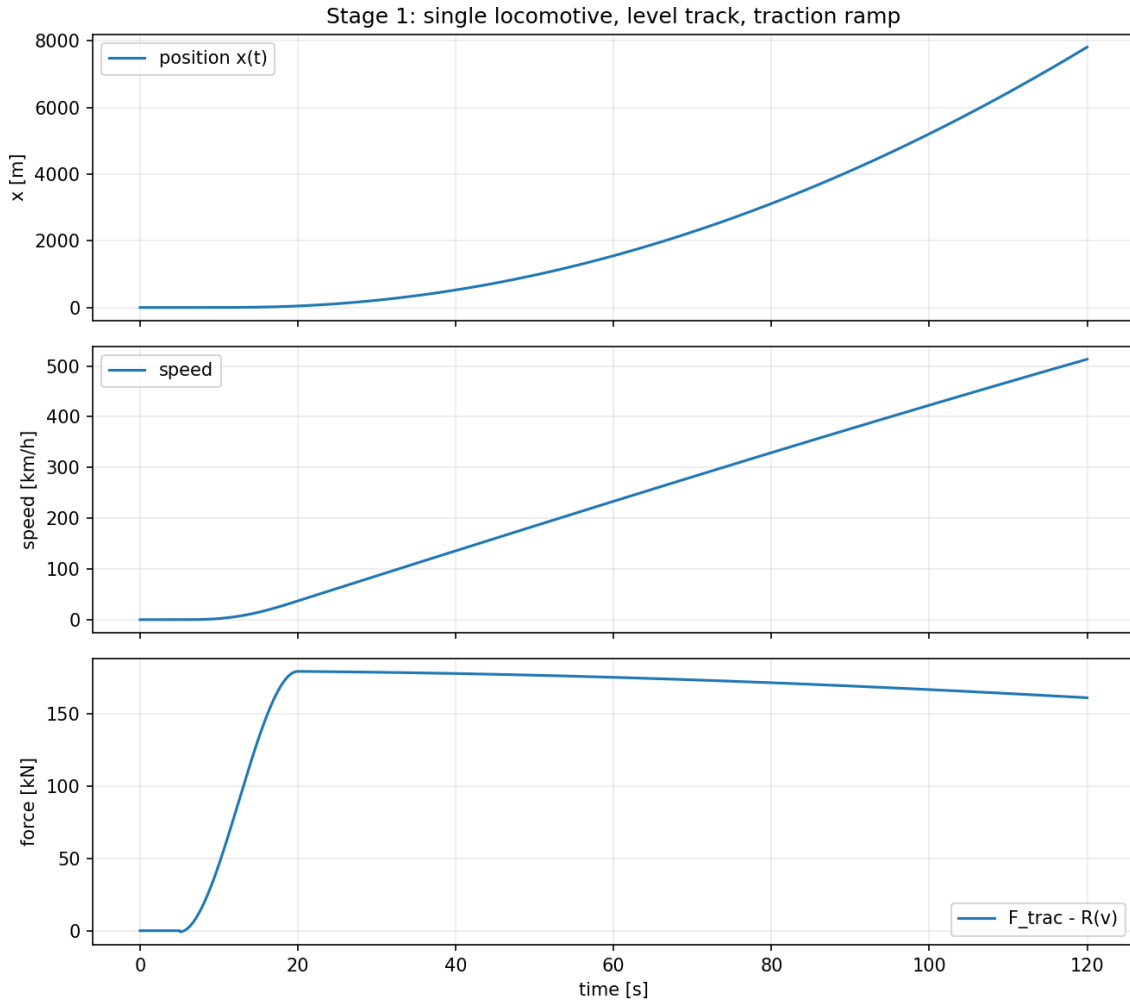


Figure 4.1: Stage 1 reference run: single vehicle on level track with a smooth traction ramp. Top to bottom: position, speed, and net longitudinal force $F_{\text{trac}} - R(v)$ versus time.

4.2 Stage 2: locomotive and one car with a linear coupler

A second mass m_1 is attached behind the lead vehicle m_0 . Nominal centre-to-centre spacing is L_0 . Coupler extension (positive in draft) is

$$\delta = (x_0 - x_1) - L_0, \quad \dot{\delta} = v_0 - v_1. \quad (4.3)$$

A linear spring-damper coupler exerts force $F = k\delta + c\dot{\delta}$ on the rear car along the direction of travel; the lead feels $-F$. With G_i and R_i evaluated per vehicle,

$$m_0\dot{v}_0 = F_{\text{trac}} - F_{\text{brk},0} - R_0(v_0) - G_0(x_0) - F, \quad (4.4)$$

$$m_1\dot{v}_1 = -F_{\text{brk},1} - R_1(v_1) - G_1(x_1) + F. \quad (4.5)$$

The state vector is (x_0, x_1, v_0, v_1) . For stiff (k, c) the two speeds track almost identically on a speed-time plot even though the coupler force and extension carry the load path; fig. 4.2 shows the characteristic transient in F and δ as slack is taken up and the train settles into quasi-uniform acceleration under traction.

4.3 Stage 3: nonlinear coupler with slack and asymmetric buff/draft

The linear model is retained for comparison, but the simulated dynamics replace F by a piecewise law with slack half-width s . For $|\delta| \leq s$ the coupler force is zero; for $\delta > s$ (draft) stiffness and damping (k_d, c_d) apply to the over-travel $\delta - s$, and for $\delta < -s$ (buff) a possibly different pair (k_b, c_b) applies to $\delta + s$. The same two-body equations eq. (4.5) hold with this $F(\delta, \dot{\delta})$. Figure 4.3 overlays the *linear* force $k\delta + c\dot{\delta}$ that would correspond to Stage 2 parameters against the *actual* nonlinear force along the trajectory, together with δ and $v_0 - v_1$. The deadband produces intervals of near-zero force and sharper loading when contact engages, which motivates slack-aware modelling for impact-style longitudinal loads.

4.4 Stage 4: three bodies and sequential coupler forces

With vehicles indexed 0, 1, 2 front to rear, coupler $j \in \{0, 1\}$ connects j to $j + 1$ and F_j denotes the force on the *rear* vehicle in that pair. The recursions generalise naturally:

$$m_0\dot{v}_0 = F_{\text{trac}} - R_0 - G_0 - F_0, \quad (4.6)$$

$$m_1\dot{v}_1 = -R_1 - G_1 + F_0 - F_1, \quad (4.7)$$

$$m_2\dot{v}_2 = -R_2 - G_2 + F_1, \quad (4.8)$$

with each F_j obtained from the nonlinear coupler law applied to $(\delta_j, \dot{\delta}_j)$ where $\delta_j = (x_j - x_{j+1}) - L_{0,j}$. End-to-end stretch relative to nominal spacing, $(x_0 - x_2) - (L_{0,0} + L_{0,1})$, summarises how the consist elongates in draft. Figure 4.4 shows speeds, both coupler forces, and stretch for a traction-only run. An optional variant applies a time-localised service-brake force on the lead only while traction continues, illustrating redistribution of draft forces (fig. 4.5).

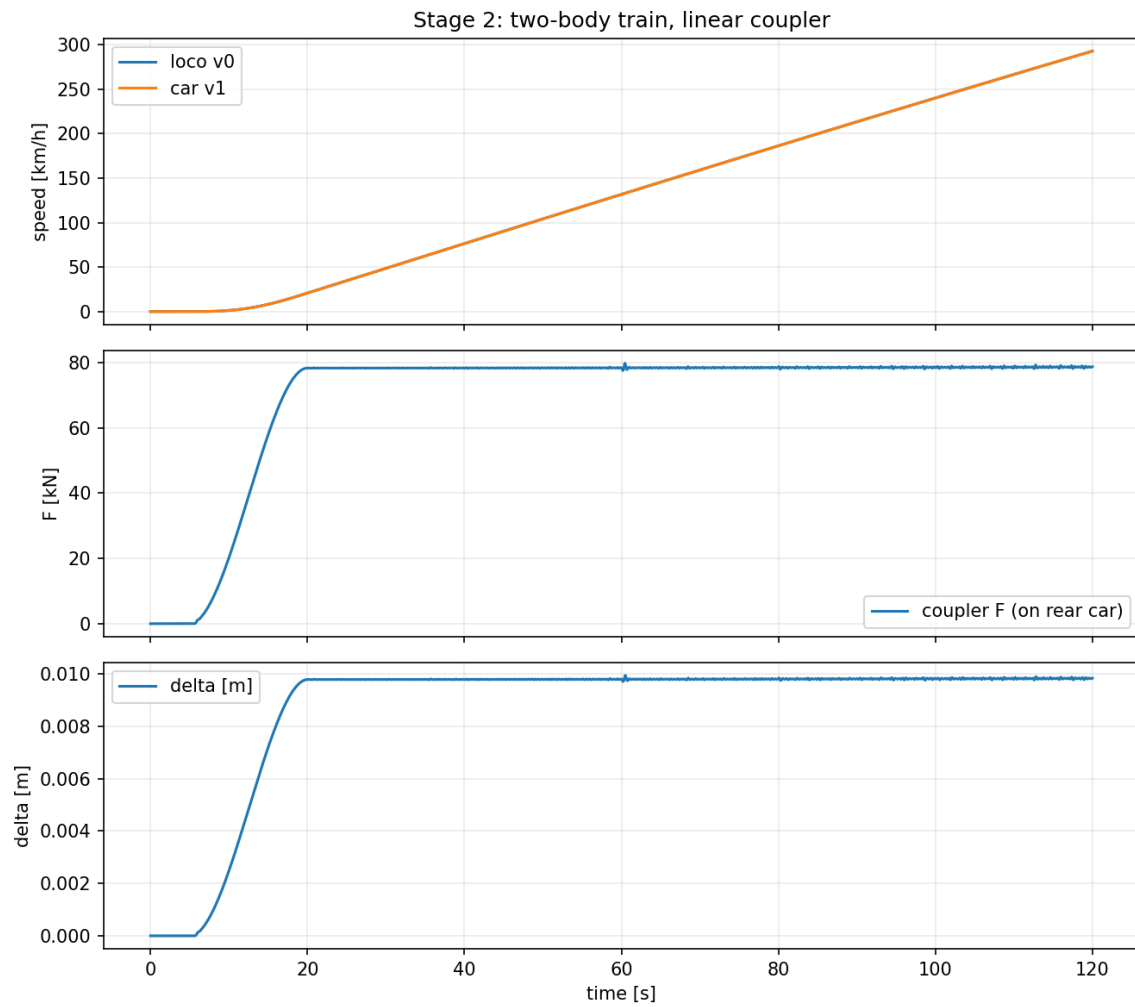


Figure 4.2: Stage 2: two-body train with linear coupler. Speeds of lead and trailing vehicle (overlapping at this scale), coupler force on the rear car, and extension δ .

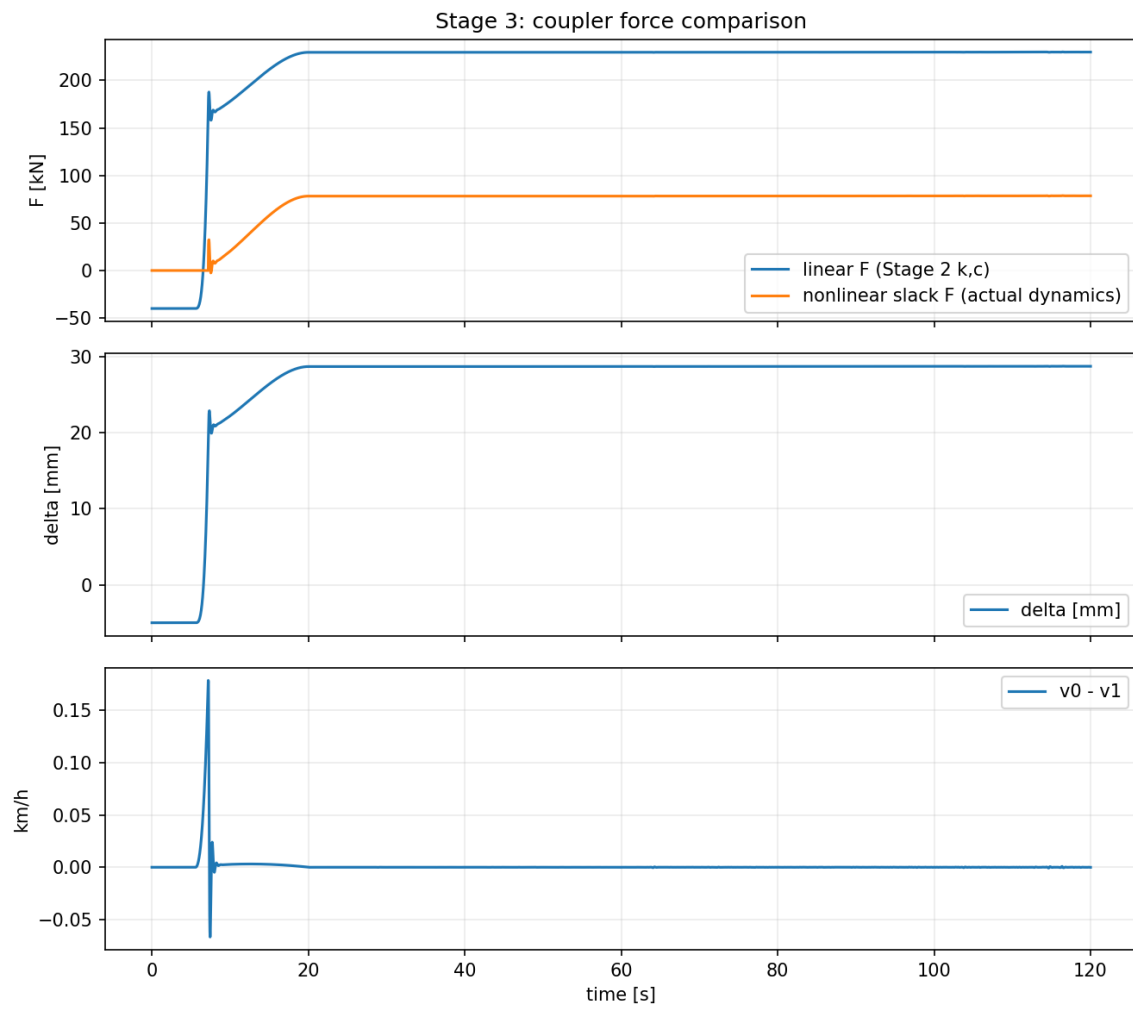


Figure 4.3: Stage 3: comparison of linear spring-damper force (Stage 2 parameters) versus nonlinear slack coupler force along the same trajectory, with extension and relative speed.

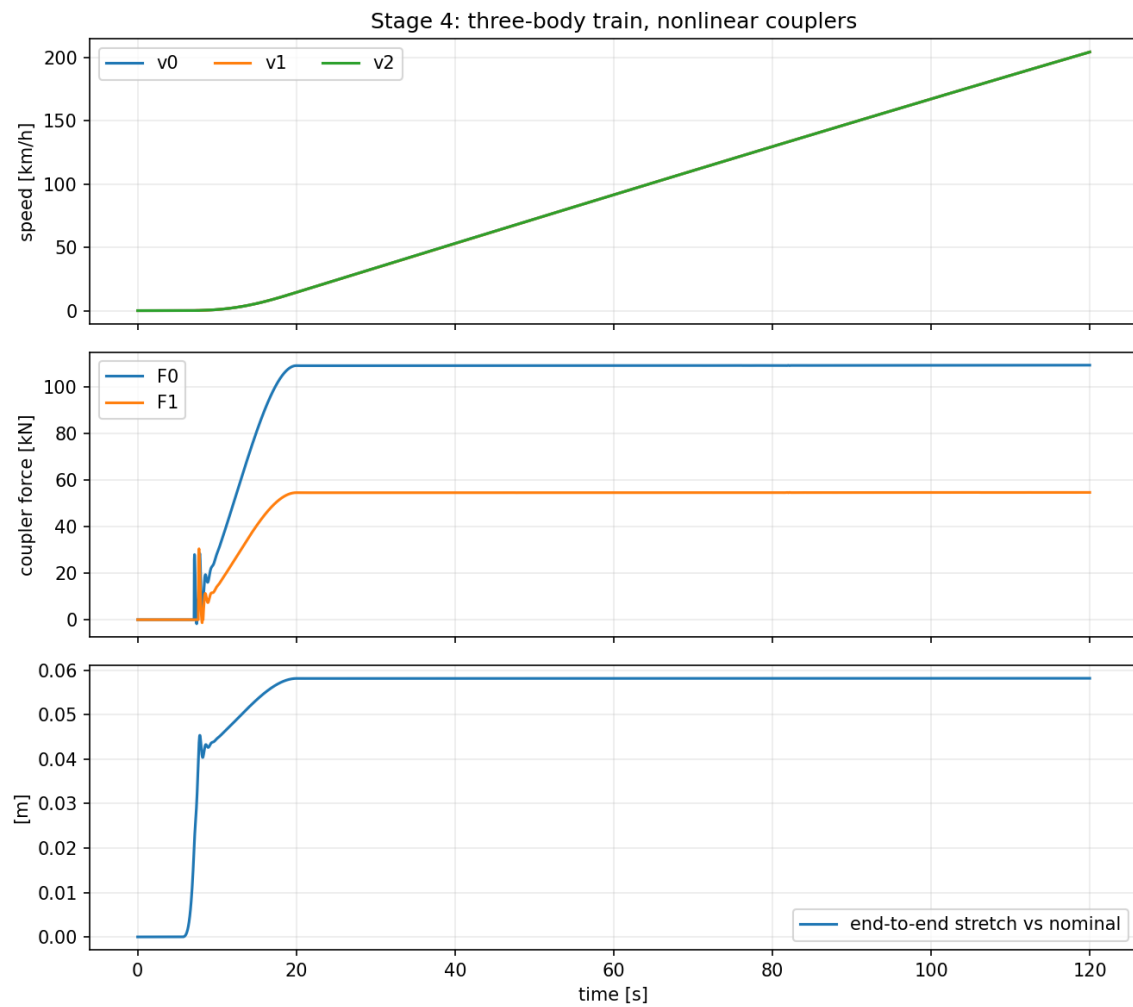


Figure 4.4: Stage 4: three-body train with nonlinear couplers. Speeds, coupler forces F_0 and F_1 , and end-to-end stretch versus time.

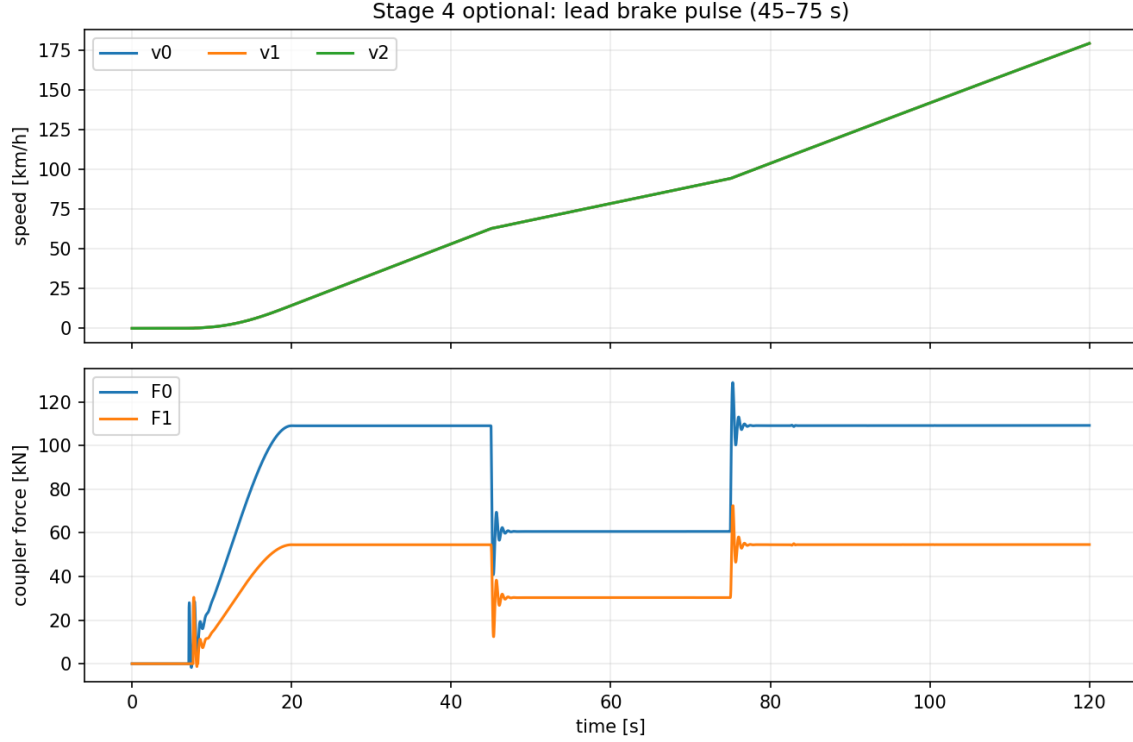


Figure 4.5: Stage 4 optional scenario: lead-only brake pulse; speeds and coupler forces.

4.5 Stage 5: general N -vehicle assembly

The state is stacked as $\mathbf{y} = (x_0, \dots, x_{N-1}, v_0, \dots, v_{N-1})^\top \in \mathbb{R}^{2N}$. For each coupler $j = 0, \dots, N-2$, δ_j and $\dot{\delta}_j$ are defined as in eq. (4.3) with vehicle j ahead of $j+1$. The force F_j on the rear vehicle follows the slack model with per-coupler parameters. The right-hand side for vehicle i is

$$m_i \dot{v}_i = F_{\text{trac},i} - F_{\text{brk},i} - R_i(v_i) - G_i(x_i) - C_i + F_{i-1}^{\text{in}} - F_i^{\text{out}}, \quad (4.9)$$

with the convention $F_{-1}^{\text{in}} = F_{N-1}^{\text{out}} = 0$, and with traction and braking possibly zero on unpowered cars. Data structures (vehicle and coupler parameter records, and a flat route profile) group masses, Davis triples, coupler geometry, and slack/stiffness data for reuse. This is also the stage where the simulator stops being tied to a fixed train length: the same code path handles any N , which is what lets it later generate training data across a wide range of consist sizes rather than one fixed configuration. Figure 4.6 plots selected speeds and coupler forces for a consist with one powered lead and identical trailing cars; fig. 4.7 shows a space-time style heatmap of all coupler forces, illustrating the monotonic decay of steady draft force from head to tail when only the lead provides traction.

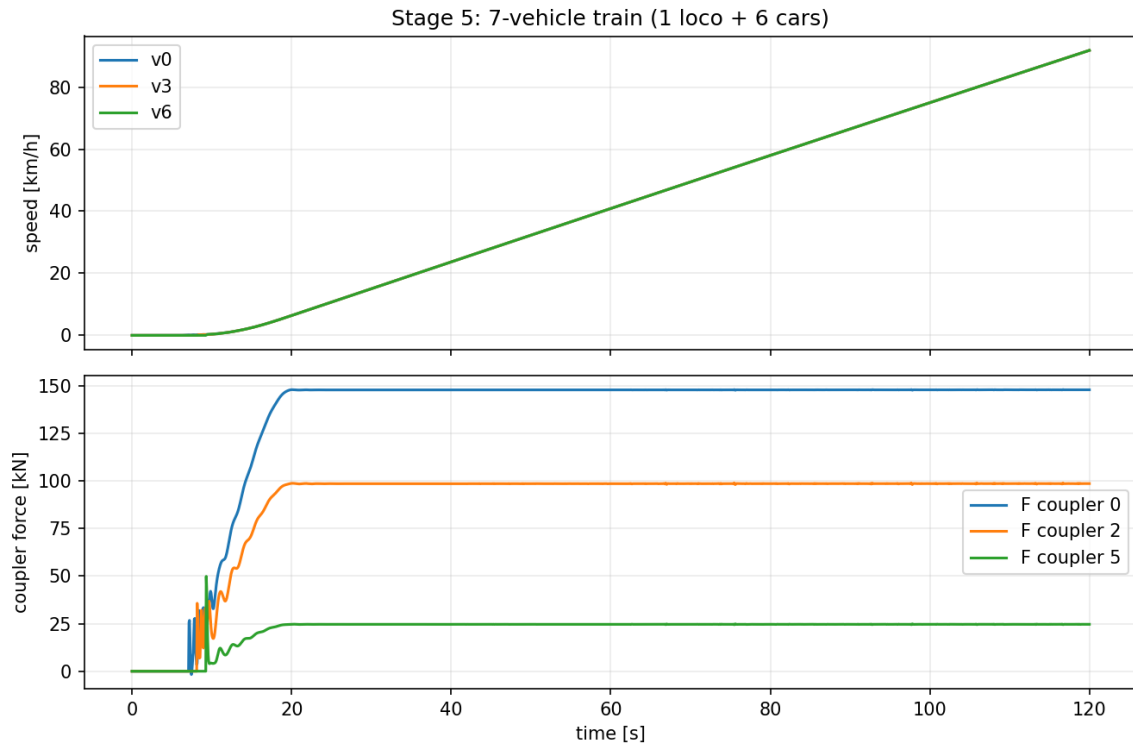


Figure 4.6: Stage 5: N -vehicle train; selected speeds and coupler forces versus time.

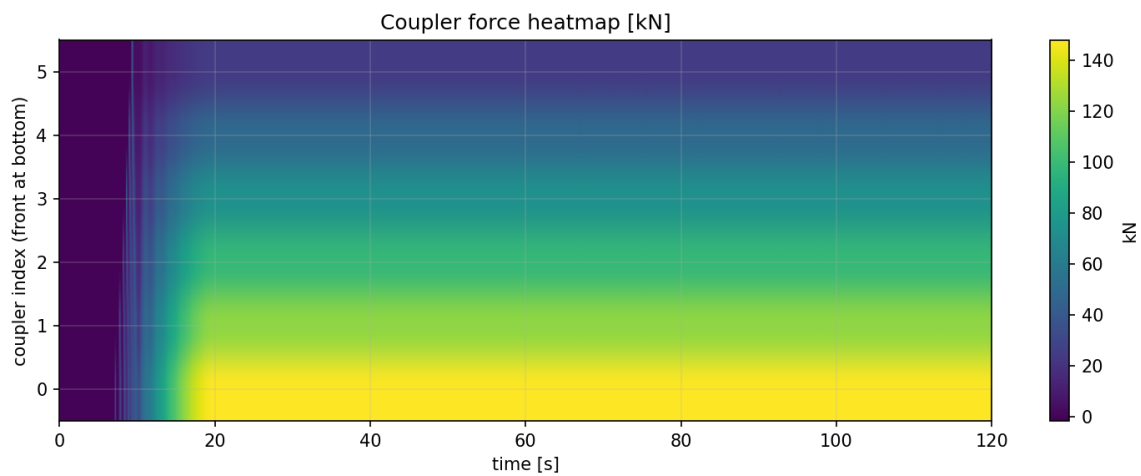


Figure 4.7: Stage 5: coupler force heatmap (force versus time and coupler index).

4.6 Stage 6: per-vehicle grade and curvature from a route profile

Grade and curvature are no longer evaluated at a single global x ; each vehicle samples $\sin\theta(x_i)$ and optionally $\kappa(x_i)$ at its own position. The profile is stored as piecewise-linear nodes and interpolated (e.g. linearly in arc length). Then $G_i = m_i g \sin\theta(x_i)$ and the same curvature proxy as before applies per vehicle. This introduces spatial dispersion along the train: the head and tail can experience different grades simultaneously, which excites relative motion and coupler cycling even when speed traces remain nearly overlaid. Figure 4.8 shows the prescribed $\sin\theta$ along route distance in the top panel and the time evolution of lead and rear speed and selected coupler forces below; the horizontal axes of the lower panels are simulation time, not chainage, so the spatial profile should be read as a fixed map through which the train moves.

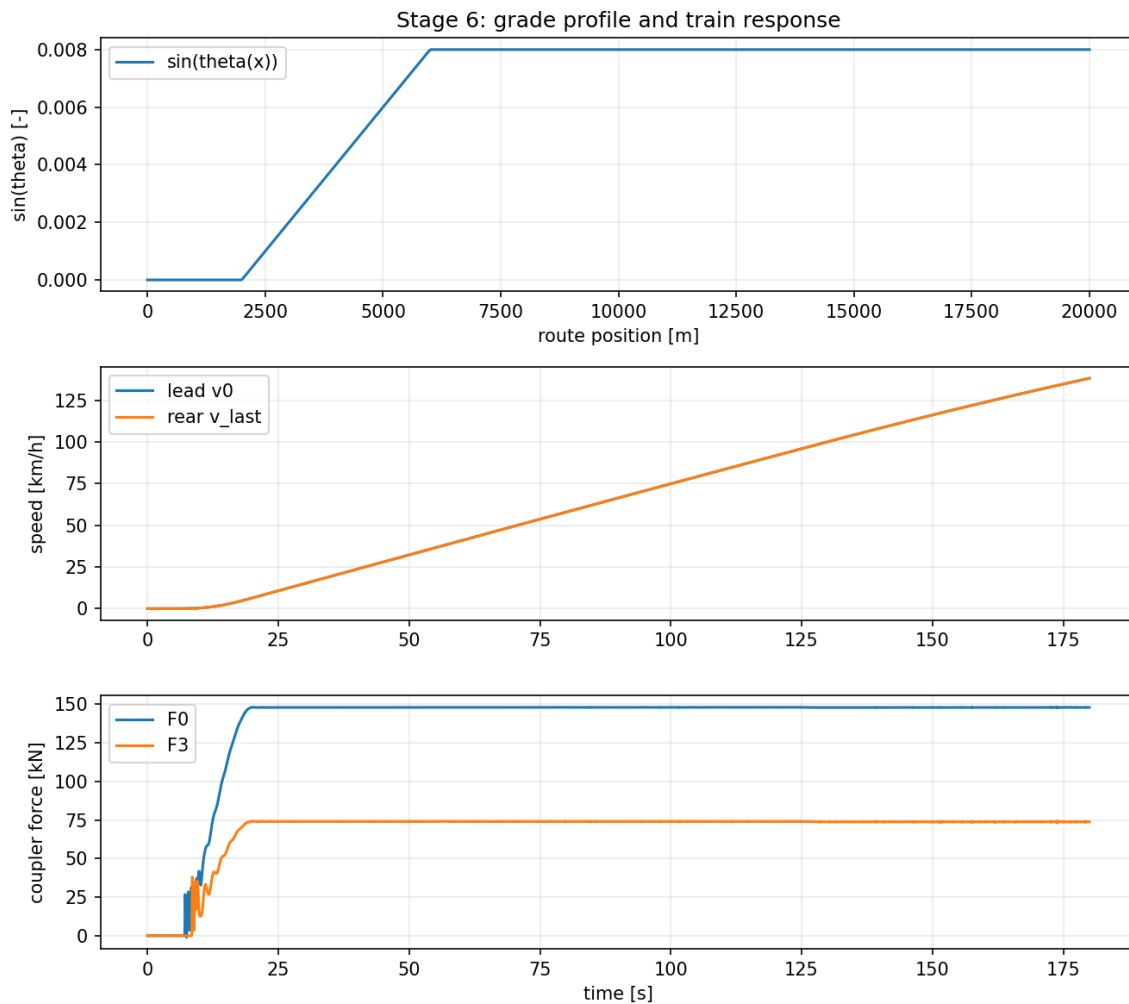


Figure 4.8: Stage 6: interpolated grade profile $\sin\theta$ versus route position, with lead and rear speeds and coupler forces versus simulation time.

4.7 Stage 7: actuator dynamics and power-limited traction

The state is augmented by first-order lag variables for commanded brake level $z_{\text{brk},i}$ and commanded traction level $z_{\text{trac},i}$ on each vehicle, driven by exogenous commands $u_{\text{brk},i}(t)$ and $u_{\text{trac},i}(t)$:

$$\dot{z}_{\text{brk},i} = \frac{u_{\text{brk},i} - z_{\text{brk},i}}{\tau_b}, \quad \dot{z}_{\text{trac},i} = \frac{u_{\text{trac},i} - z_{\text{trac},i}}{\tau_t}. \quad (4.10)$$

Achieved brake force is capped by a vehicle limit, $F_{\text{brk},i} = \min(\max(z_{\text{brk},i}, 0), F_{\text{brk,max},i})$, and is applied *against the direction of travel* rather than in a fixed direction:

$$F_{\text{brk},i}^{\text{applied}} = F_{\text{brk},i} \tanh\left(\frac{v_i}{\varepsilon_b}\right), \quad (4.11)$$

with ε_b a small blend width. The tanh is preferred to $\text{sgn}(v_i)$ because it keeps the right-hand side smooth through $v_i = 0$, which both eases the integrator and leaves the model differentiable for the learned surrogate; at operating speed the factor is unity to machine precision. Without this term a vehicle braked to a standstill is accelerated in reverse without bound, since the brake would keep contributing a negative force at $v_i = 0$. For powered vehicles, achieved traction is the continuous power cap

$$F_{\text{trac},i} = \min\left(\max(z_{\text{trac},i}, 0), \frac{P_{\text{max}}}{\max(|v_i|, \varepsilon)}\right), \quad (4.12)$$

with P_{max} an illustrative constant power ceiling and ε a small speed regulariser. These substitutes replace instantaneous step forces in eq. (4.9). Figure 4.9 shows lead speed together with the lead's z_{brk} and z_{trac} for a scenario that overlaps traction command with braking command; the plotted states should not be confused with achieved $F_{\text{trac},i}$ and $F_{\text{brk},i}$, which also reflect eq. (4.12) and the brake saturation. This stage is what makes the control commands in chapter 3 realistic rather than instantaneous; a full driver model or anti-slip logic is outside the present scope.

4.8 Stage 8: scope relative to industrial high fidelity

This build-up closes with a structured discussion of effects commonly associated with “high fidelity” in operational or research simulators: hysteretic draft gear, pneumatic brake pipe dynamics, distributed traction and adhesion limits, stochastic parameters, and calibration to field data. Those modules are not part of the reference code path summarised here; they mark the boundary between the intermediate-fidelity simulator used for learning experiments in this thesis and a fully industrial toolchain. Turning this working right-hand side into a small, reusable simulation function happens next, in the chapter where the surrogate model is built and connected to experiments.

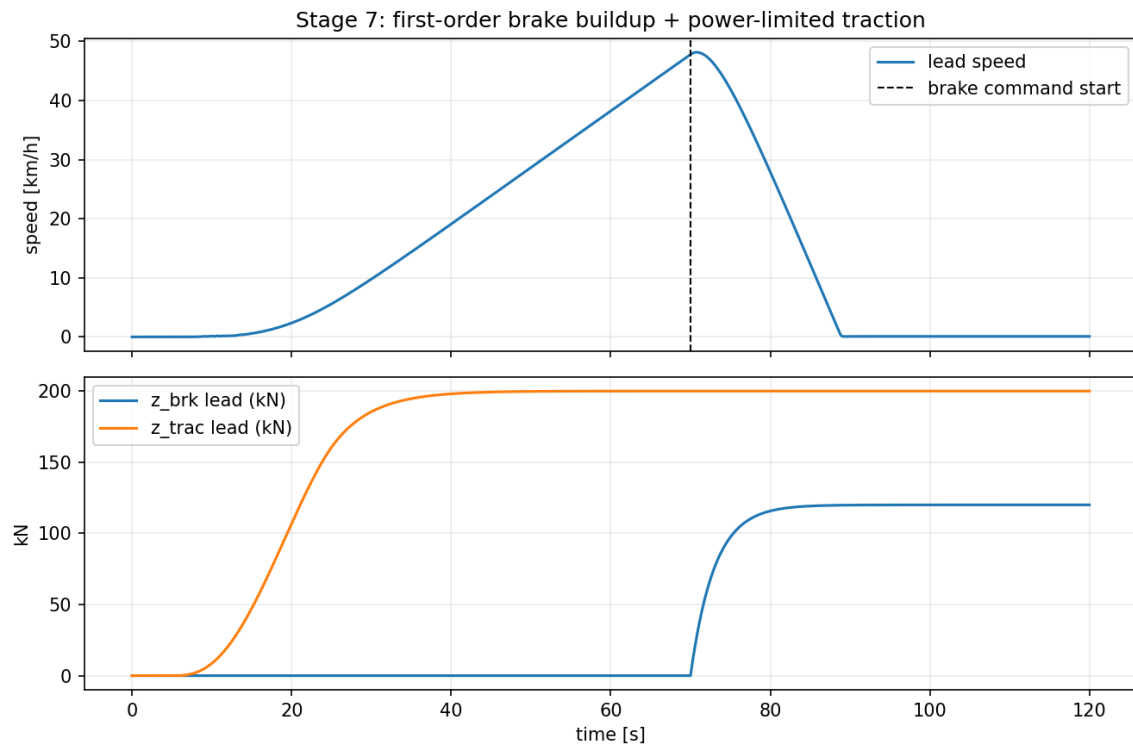


Figure 4.9: Stage 7: lead speed with brake-command onset marker; first-order lag states z_{brk} and z_{trac} on the lead versus time.

4.9 Chapter summary

This chapter documented the reference simulator, built up one physical effect at a time: a single vehicle under traction and resistance, a linear coupler, a nonlinear slack coupler with asymmetric buff and draft, a general chain of N vehicles, per-vehicle grade and curvature sampled from a route profile, and actuator lag with power-limited traction. The result is a reproducible, intermediate-fidelity simulator that matches the multi-body, route-coupled structure of the problem stated in chapter 3, and that can generate consist configurations, routes, and control plans in the volume needed to train a surrogate model. The next chapter uses this simulator to build and train that surrogate.

Chapter 5

Route Acquisition and Terrain Data Pipeline

Section 3.2 defined the route profile $\mathcal{R}$ as a piecewise field of grade, curvature, and speed limit sampled along a track. Chapter 4 then treats that field as an input it is simply handed. This chapter explains where that input actually comes from.

One of the locked decisions behind this thesis is that routes are built from real terrain rather than invented from scratch (section 3.2). The reasoning is straightforward: a surrogate model is only as convincing as the data it is trained on, and a grade profile that looks nothing like a real railway line is a weak foundation for later claims about generalization. Satisfying that decision, however, is not a small task. It requires pulling raw track geometry from an open mapping database, turning that geometry into clean, continuous corridors, attaching real elevation data to it, and converting the result into the exact numerical fields the simulator and surrogate expect. This chapter documents the pipeline that does that.

The pipeline is organized as four stages plus an export step, each with its own storage folder and its own tab in an accompanying application:

acquire $\rightarrow$ resample $\rightarrow$ elevation $\rightarrow$ profile $\rightarrow$ export

It is built to run in batches: a route is fetched and checked, good routes are collected, and a dataset is generated for that batch, rather than attempting to process an entire national rail network in one pass.

Figure 5.1: Overview of the route acquisition pipeline, from raw OpenStreetMap track data to the route-tensor fields consumed by the reference simulator.

5.1 Stage 1: Acquisition

5.1.1 Why acquisition has to be area-first

Railway lines are not tidy, self-contained objects in OpenStreetMap (OSM). Track is mapped as many short, disconnected segments called ways, and while OSM does define route relations

that are supposed to group these ways into a single named line, coverage for United States freight corridors is incomplete and a given relation does not reliably correspond to one clean, walkable corridor. Waiting for well-formed relations to exist would leave large parts of the network unusable.

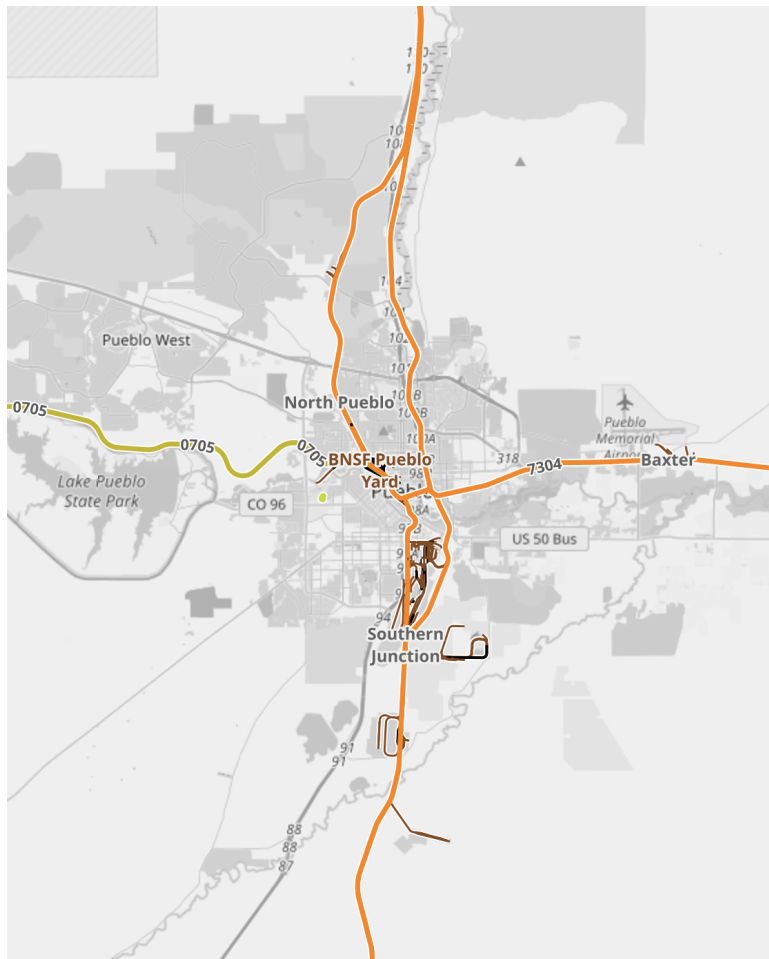


Figure 5.2: Railway mapping around Pueblo, Colorado, rendered from OpenStreetMap track data. Mainline corridors (orange) run through a dense tangle of yard and service track (brown) and meet at several junctions. There is no single map object corresponding to “the line through Pueblo”; corridors have to be recovered from the segments.

The pipeline therefore treats selection as area-first. A user specifies a bounding box or a radius around a point, all track ways inside that area are pulled, and the resulting tangle of segments is split back into corridors algorithmically rather than relying on OSM’s own grouping. Fetching by relation ID is kept as a secondary path for the cases where a good relation happens to exist.

5.1.2 Corridor extraction as a graph problem

Once an area’s track ways are pulled, they are assembled into a graph whose vertices are the endpoints and junctions of the track and whose edges are the individual way segments.

Every vertex falls into one of three roles: a *continuation* node has degree two and simply passes a line through; a *line-end* has degree one and marks where a piece of track terminates; a *junction* has degree three or higher and marks a real branch point. Corridors are then extracted as maximal paths through continuation nodes, breaking wherever a line-end or a junction is reached. This is what turns an unordered pile of map fragments into a set of distinct, traceable lines.

For the radius-based query mode, an optional bounded outward crawl extends this further: starting from the area first pulled, the pipeline follows each line's open ends past the edge of the original radius, issuing additional queries just beyond them, until each line reaches a natural end or a hard cap on distance, iteration count, or total ways is hit. This captures lines that only pass through the queried area rather than starting or ending inside it, while the cap keeps the crawl from expanding into the fully connected national network.

5.1.3 Putting fragments back together

OSM splits a single physical rail line into many ways wherever a junction, a bridge, or a change in tagging occurs, and sidings introduce additional junctions that fragment a line even further. A long corridor can therefore arrive as dozens of disconnected pieces. A grouping step re-links collinear fragments, across both real junctions and small unshared endpoint gaps, using endpoint proximity together with heading continuity (whether the fragments point the same direction where they meet). Nothing is merged or discarded in this step; fragments keep their identity and remain reversible, they are simply labeled with a shared line identifier and an order along that line, so later stages can stitch them into one ordered polyline. Ways tagged as sidings, spurs, yards, or crossovers are flagged as service track and kept out of this mainline grouping, though they can still be included in a batch if a user specifically wants them.

A related routine builds a robust centerline directly from a set of member ways, whether those ways come from a relation fetch or from a grouped corridor. It stitches unordered and reversed ways into one ordered polyline using shared endpoint nodes, with a coordinate-based fallback when node identifiers do not line up, and it explicitly detects gaps (gaps in space) and branches (points where more than one continuation is possible) rather than silently picking one. Duplicate junction vertices are removed, per-vertex tags such as speed limit, bridge, tunnel, cutting, and embankment are carried along, and geodesic arc length is computed for the whole line.

5.1.4 Interactive selection

An accompanying Streamlit application exposes both acquisition modes. In area mode, a user enters a bounding box, the app fetches and extracts all corridors in it, and shows them color-coded on a map alongside a sortable table and per-corridor quality checks (gap count and size, branch points, how much of the line has known speed limits, and what fraction runs over bridges or through tunnels). Corridors selected from this view are added to a batch for export. In relation mode, a single named route is fetched and inspected the same way. For the common case of just wanting a handful of routes near a particular place, the simplest path is radius mode with the outward crawl enabled: a user clicks a line on the map, its

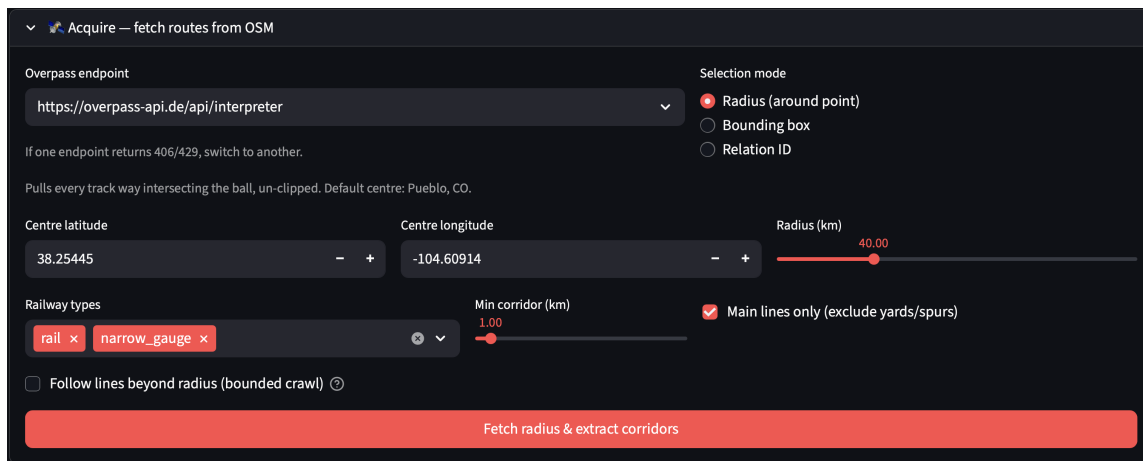


Figure 5.3: The acquisition panel of the route application. A selection mode (radius, bounding box, or relation identifier) is chosen on the right; the centre point, radius, admissible railway types, and a minimum corridor length are set on the left. The mainline-only filter excludes yards and spurs, and the bounded outward crawl described in section 5.1 is toggled at the bottom. Several Overpass endpoints are selectable because any one of them may be rate-limiting at a given moment.

full extent highlights, and it is written to storage with every vertex’s structural flags and speed-limit coverage already attached.

5.2 Stage 1.5: Resampling to a uniform grid

OSM digitizes track adaptively with respect to horizontal curvature: dense vertices on curves, sparse vertices on straight tangents. That density pattern is a poor match for what the next stage needs, since it under-samples exactly the straight, low-curvature stretches where the vertical profile, grade, still needs to be captured accurately. The resampling stage rewrites each stored route onto a fixed arc-length grid (10 meter spacing by default) by linearly interpolating longitude and latitude along the line’s arc length.

Two details matter here. First, step-valued features, speed limit, way identifier, and the bridge, tunnel, cutting, and embankment flags, are not smoothly interpolated; each resampled point simply inherits the value from whichever source segment it falls inside, and a sample is inserted exactly at the point where such a feature changes, so a speed-limit step or a bridge entrance is not blurred across a grid cell. Second, the resampling step validates its own output: coincident vertices are dropped, arc length is enforced to be strictly increasing, and each point is tagged with the length of the source segment it came from and a flag marking whether it was interpolated across an unusually long or bridged gap in the original data.



Figure 5.4: Inspection view for a resampled route: a 49.1 km corridor rewritten onto a uniform 10 m grid, yielding 4920 points. The plan view distinguishes points that sit on an original map vertex from those interpolated across long source segments (713 of them here), so stretches where the geometry is inferred rather than mapped remain visible downstream.

5.3 Stage 2: Elevation

Each resampled point is next given a ground elevation from the USGS 3DEP digital elevation model (DEM), producing a three-dimensional route as a sequence of longitude, latitude, and elevation triples. Elevation is queried in batches against 3DEP’s image-service endpoint, since it accepts many points per request, with a slower per-point service used as a fallback for any vertex the batch call cannot resolve. Requests retry with backoff when the service is busy or briefly unavailable. Any point elevation cannot be determined for is left as a missing value rather than guessed, and every earlier per-vertex field from acquisition and resampling is preserved alongside the new elevation data.

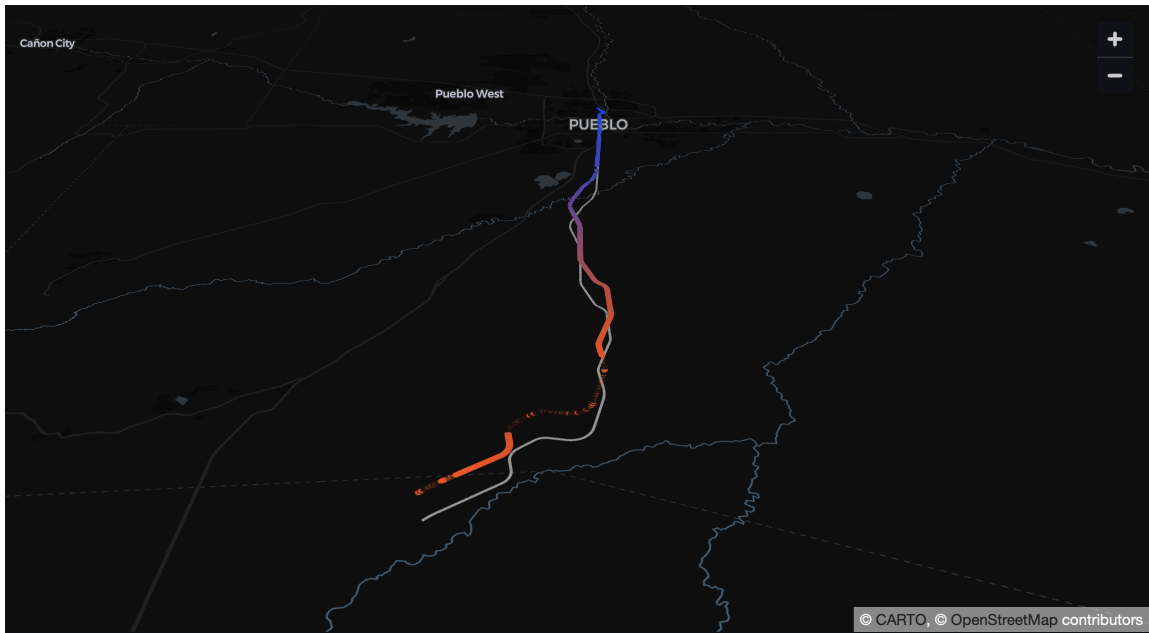


Figure 5.5: An elevated route rendered in three dimensions, coloured by elevation along the line, with the mapped ground track shown alongside. Visual inspection at this stage catches corridors that have been stitched incorrectly or that jump between parallel alignments, before any derivative is taken.

5.4 Stage 3: Deriving the route profile

This is the stage that produces the exact fields the reference simulator and the surrogate consume: the grade field $\sin \theta(s)$, the curvature proxy $\kappa(s)$, and the speed limit $v_{\max}(s)$ introduced in section 3.2.

5.4.1 Correcting for bridges and tunnels

On a bridge or inside a tunnel, the DEM reports the elevation of the structure or the ground above it, not the elevation of the rail itself, and it sometimes reports no value at all. These

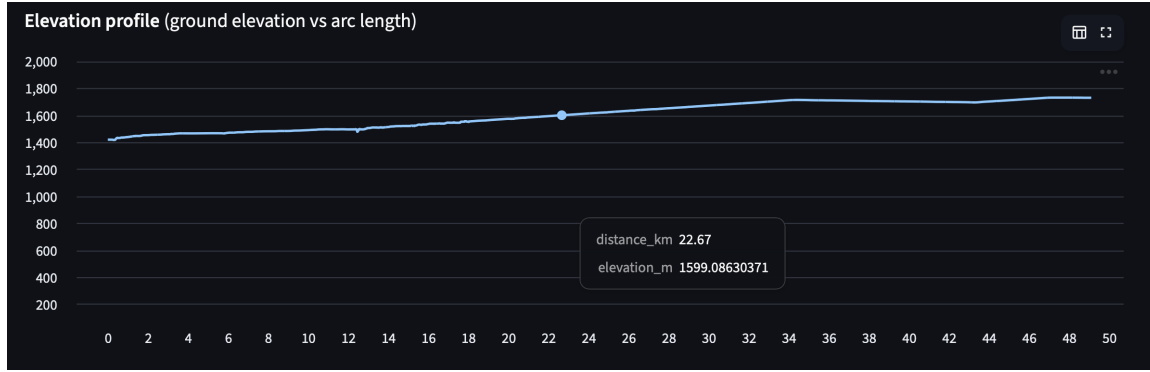


Figure 5.6: Ground elevation against arc length for the same corridor, climbing from roughly 1430 m to 1740 m over 49 km. This is the raw signal from which grade is derived; the metre-scale error it carries is negligible against the elevation range plotted here but is large against the elevation *change* between adjacent 10 m samples, which is what motivates the smoothing described below.

stretches are identified using the structural flags carried since acquisition, discarded, and filled in by linear interpolation across the gap.

5.4.2 Smoothing and differentiating together

Raw DEM elevation carries roughly a meter of error, which is small compared to elevation itself but large compared to elevation’s rate of change. Differentiating the raw signal directly would produce a grade estimate that is almost entirely noise. Instead, a Savitzky-Golay filter is applied, which fits a local polynomial through a moving window of points and yields a smoothed elevation value and its analytic derivative from the same operation, rather than smoothing and differentiating as two separate, error-compounding steps. Grade is then computed as $\sin \theta = \sin(\arctan(dz/ds))$, with a physically motivated clamp on the maximum grade to guard against any remaining artifacts.

Curvature is derived the same way. The horizontal coordinates are projected into a local planar frame, and the first and second Savitzky-Golay derivatives of that planar path give

$$\kappa = \frac{x'y'' - y'x''}{(x'^2 + y'^2)^{3/2}}, \quad (5.1)$$

clamped to a minimum turning radius for the same reason grade is clamped.

5.4.3 Speed limit and output

Speed limit is converted from the map’s tagged value into meters per second where available, and falls back to a Federal Railroad Administration track-class-based estimate where OSM coverage is missing. The finished profile is written in two forms: a GeoJSON file for visual inspection, and a compact array file holding exactly the fields `route_s`, `route_sin_theta`, `route_kappa`, and `route_vmax`, matching the route-tensor layout used for dataset generation (chapter 4). As a final check, the array file is used to construct an actual route-profile object and sample it, confirming the file behaves correctly before it is trusted for later stages.

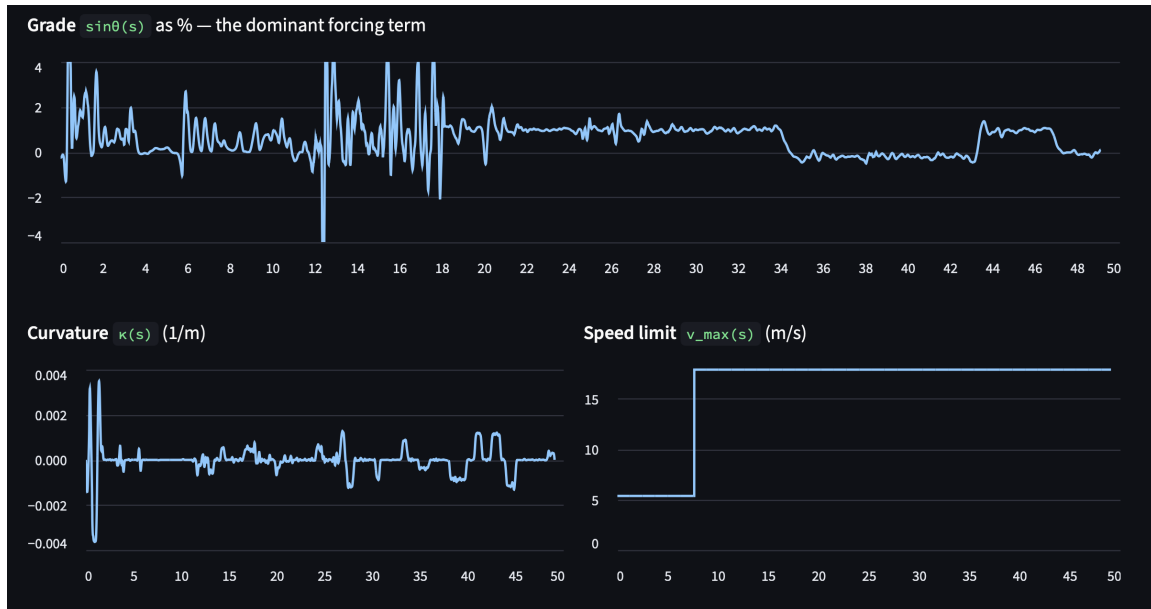


Figure 5.7: The three derived fields the simulator consumes, plotted against chainage in kilometres: grade $\sin\theta(s)$ as a percentage (top), the curvature proxy $\kappa(s)$ (bottom left), and the speed limit $v_{\max}(s)$ (bottom right). The noisier grade over the first 20 km is the urban approach, where frequent bridges, junctions, and short mapped segments degrade the elevation signal; beyond it the line settles onto a sustained climb near one percent. The speed limit is a step field, here rising where the line leaves restricted urban trackage.

5.5 Export

Finished routes are collected into a single export bundle: one array file per route, an index summarizing each route's location, endpoints, length, and grade and curvature statistics, and a manifest recording what each channel means and how the route was produced. This bundle is the handoff point between route acquisition and dataset generation; its array files drop directly onto the reference simulator's route-profile input with no further conversion.

5.6 Keeping large queries from failing

Pulling all track in a large or dense area from OpenStreetMap's Overpass query service in one request tends to time out. Three measures address this and are on by default. Queries exclude service track and industrial, tourism, and military spurs, which both matches the intended scope of freight main lines and substantially reduces the volume of data returned in dense metro areas. The initial seed query for a crawl is split into a grid of small requests rather than one large one, with the outward crawl then expanding in small steps from there. Every request retries on timeout or server error with exponential backoff and jitter, and rotates across known mirror servers when one is unavailable.

5.7 Two ways to build a dataset

For a handful of routes, the interactive application is enough: fetch an area, inspect the corridors, click the ones that look right, and export. For a larger dataset, the recommended path is to first verify the pipeline on a few known areas interactively, checking that lines render as continuous single corridors, gaps are short and located at bridges, branch points sit at genuine junctions, and a corridor's traced length is plausible against an independent map. Once that trust is established, a headless batch runner crawls a curated list of seed points into a deduplicated dataset. Lines pulled from overlapping seeds are deduplicated by their shared underlying map identifiers, keeping the longest version of any line pulled more than once, and the whole run is resumable from a cache so a full national-scale pull does not have to restart from nothing after an interruption.

5.8 Testing and design choices

The line-stitching logic, the part of the pipeline responsible for turning disordered, possibly reversed map fragments into one correct ordered corridor, is unit tested offline against a fixture built from a captured query response, so its correctness does not depend on network access or on live map data staying unchanged.

A few choices run through the whole pipeline. Acquisition and assembly are pure functions wherever they do not require a live network call, which is what makes offline testing possible in the first place. Data quality problems are surfaced rather than silently patched: gaps are bridged but their size is reported, branch points are flagged rather than resolved automatically, and disconnected fragments are dropped and listed rather than dropped quietly,

so a user decides whether a given route is clean enough to use. Finally, the core pipeline, acquisition, resampling, elevation fusion, and profile derivation, depends only on NumPy and an HTTP request library; the Savitzky-Golay smoothing and differentiation is a small NumPy implementation of its own rather than a dependency on a larger scientific computing package, so the whole path can run and be tested without a heavier install.

5.9 Chapter summary

This chapter documented the pipeline that turns raw OpenStreetMap track data and public elevation data into the route profiles this thesis relies on: area-based acquisition and graph-based corridor extraction, fragment stitching back into whole lines, resampling onto a uniform grid, elevation fusion, and Savitzky-Golay-based derivation of grade, curvature, and speed limit. The output, arrays of `route_s`, `route_sin_theta`, `route_kappa`, and `route_vmax`, is exactly what the reference simulator in chapter 4 consumes and what later chapters use to generate training data for the surrogate. Because this pipeline draws on real corridors rather than invented ones, it is also what makes it possible to hold out whole real corridors for the generalization experiments described in later chapters, rather than only holding out synthetic variations of the same underlying route.

Chapter 6

RailLab: A Desktop Research Console

Chapter 4 built the reference simulator and chapter 5 built the pipeline that feeds it real terrain. Both are libraries: they are driven from notebooks and scripts, and everything they need, a consist, a route, a set of driving commands, arrives as Python objects assembled by hand. That is a workable way to produce a figure for a chapter. It is a poor way to explore, and exploration is most of what the work in the following chapters actually consists of.

This chapter documents *RailLab*, a desktop application built over the same backend. It is not a deliverable of the thesis in its own right, and none of the numerical claims in later chapters depend on it. It is a research instrument, and it is documented here for the same reason the simulator’s staging is documented: the way a tool shapes what questions get asked is part of the method.

6.1 Why a graphical console

Three problems motivated building it.

The first is the cost of a question. Asking “what happens to coupler forces if this train is thirty cars longer on this particular grade” requires, in script form, editing a consist definition, re-resolving car parameters, rebuilding a scenario, re-running the solver, and re-plotting, with the answer arriving a few minutes and several opportunities for error later. Most such questions are never asked, not because they are uninteresting but because the friction exceeds the curiosity. Collapsing that loop to a dropdown and a button changes which questions get asked at all.

The second is the gap between a number and a shape. A simulator returns arrays. The failure modes that matter here, a run-in that the coupler model resolves unphysically, a grade artifact surviving the smoothing stage in chapter 5, a control profile whose brake command overlaps traction in a way no operator would produce, are visible immediately in a plot and nearly invisible in a summary statistic. Later chapters lean on trip-level error metrics; those metrics are trustworthy only in proportion to how much of the underlying trajectory behavior has been looked at directly.

The third is reproducible bookkeeping. The experiments in later chapters need many consists run against comparatively few routes, and every artifact involved, a car type, a train, a control profile, a route, has to be nameable, saveable, and reloadable so that a run

can be repeated exactly. Building an interface around those artifacts forced them to become first-class, serializable objects in the backend rather than variables in a notebook cell, which is a benefit the headless path inherits.

6.2 Architecture

The governing constraint is that the user interface never imports the numerical stack. The backend packages, `simulator2` and `routegen`, are unchanged by the application’s existence; the application reaches them only through a thin service layer. Figure 6.1 summarizes the arrangement.

Figure 6.1: RailLab’s layering. Screens call services; services are the only modules that import the numerical backend, and they return plain data. The backend packages are unchanged by the application’s existence and continue to run headless.

The layering has three consequences that matter more than the tidiness. The backend still runs headless, so notebooks, the batch dataset generator, and the figures in this document have no dependency on the interface toolkit. The services contain no interface code at all, so each is unit-testable without a display. And the boundary is a natural insertion point: the surrogate model and the controller developed in later chapters attach as additional services without any existing screen changing.

Within the application, each screen owns one stage of the workflow and communicates through shared service objects rather than by reaching into its neighbors. The `CONSIST` and `SIMULATE` screens share one consist service, so a train edited on one appears immediately on the other; `CONTROLS` and `SIMULATE` likewise share a control-profile service. Signals from a finished run are fanned out by the window shell to the `RESULTS` screen and the status indicator, so neither screen holds a reference to the other.

Long solver calls run off the interface thread. Pressing `GO` constructs a scenario, hands it to a worker object moved onto a dedicated thread, and returns immediately; the worker emits either a result or an error message when the integration finishes. Because the reference simulator integrates the whole time span in one adaptive solver call, there is no intermediate hook to report progress against or to interrupt, and the stop control is correspondingly disabled. This is a real limitation rather than an oversight: interrupting or streaming a run requires a stepwise integrator, which the reference implementation deliberately does not use.

6.3 The workflow, screen by screen

6.3.1 Route

The route screen browses the output of chapter 5. Each processed route is listed, and selecting one loads its per-vertex arrays and renders three chainage-linked plots, elevation, grade, and curvature, alongside a summary panel: length, vertex count, elevation range, total climb and descent, grade range, maximum curvature and the corresponding minimum

radius, speed-limit range, and the fraction of the line for which the map actually supplied a speed limit. Routes lacking the compact array file the simulator consumes are listed but marked, and the simulate screen will not offer them.

That last field, speed-limit coverage, is the reason the panel exists in this form. It is a data-quality property, not a physical one, and it is exactly the sort of thing that is easy to forget when a route is loaded programmatically and impossible to overlook when it is printed next to the grade range every time the route is opened.

6.3.2 Consist

The consist screen is where trains are assembled, and it implements the three-way split the data model was designed around: a library of reusable car types, an ordered train under construction, and a shelf of saved trains (fig. 6.2).

A car type is a named template, a locomotive, a loaded grain hopper, a full water tank car, carrying mass split into tare and lading, Davis resistance coefficients, traction and braking ceilings, physical length, and a coupler type. Types are stored one per file as plain text records, so the library is inspectable and version-controllable outside the application. A train is an ordered list of references to those types, which means a train file stays small regardless of length and a correction to a car type propagates to every train that uses it.

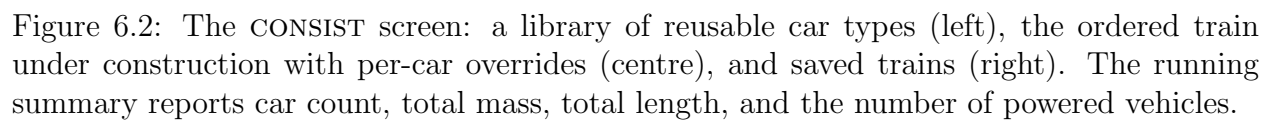
Per-car overrides are the escape hatch. Any individual car in a train may override selected fields of its type, tare mass, lading, any Davis coefficient, either force ceiling, while inheriting the rest. This makes “the same train but with one unusually heavy car” expressible without polluting the library with a near-duplicate type. A running summary along the bottom reports car count, total mass, total length, and how many vehicles are powered.

Figure 6.2 shows a consist produced by the procedural generator rather than assembled by hand: a mixed-manifest train of 133 cars, roughly 9,756 tonnes, 1698 metres long, with four powered units at the head. The generator writes through the same save path the interface uses, so generated and hand-built trains are indistinguishable downstream, which is what makes the large training sweeps of later chapters possible without a second code path.

6.3.3 Controls

The controls screen generates the driving commands. A control profile is a train-agnostic recipe: piecewise command curves expressed as a *fraction* of each vehicle’s own force ceiling rather than in newtons, so one profile drives any consist. Curves are defined by knots and interpolated with the same smoothstep shape the simulator’s traction ramp uses, which keeps commands continuous and avoids the stiffness that instantaneous steps introduce.

Profiles are generated randomly under user-specified bounds: duration, number of knots per curve, traction and brake fraction ranges, and the treatment of distributed power (fig. 6.3). Distributed power units are addressed by role, head-end, mid-train, and rear helper, resolved from each car’s label, and may be driven synchronously with a configurable lag behind the head end, independently, or not at all. The preview plots the resulting traction command for each role and the brake command below it; the visible role-to-role offset in the figure is the lag. Generation is seeded, so a profile is reproducible from its configuration alone.



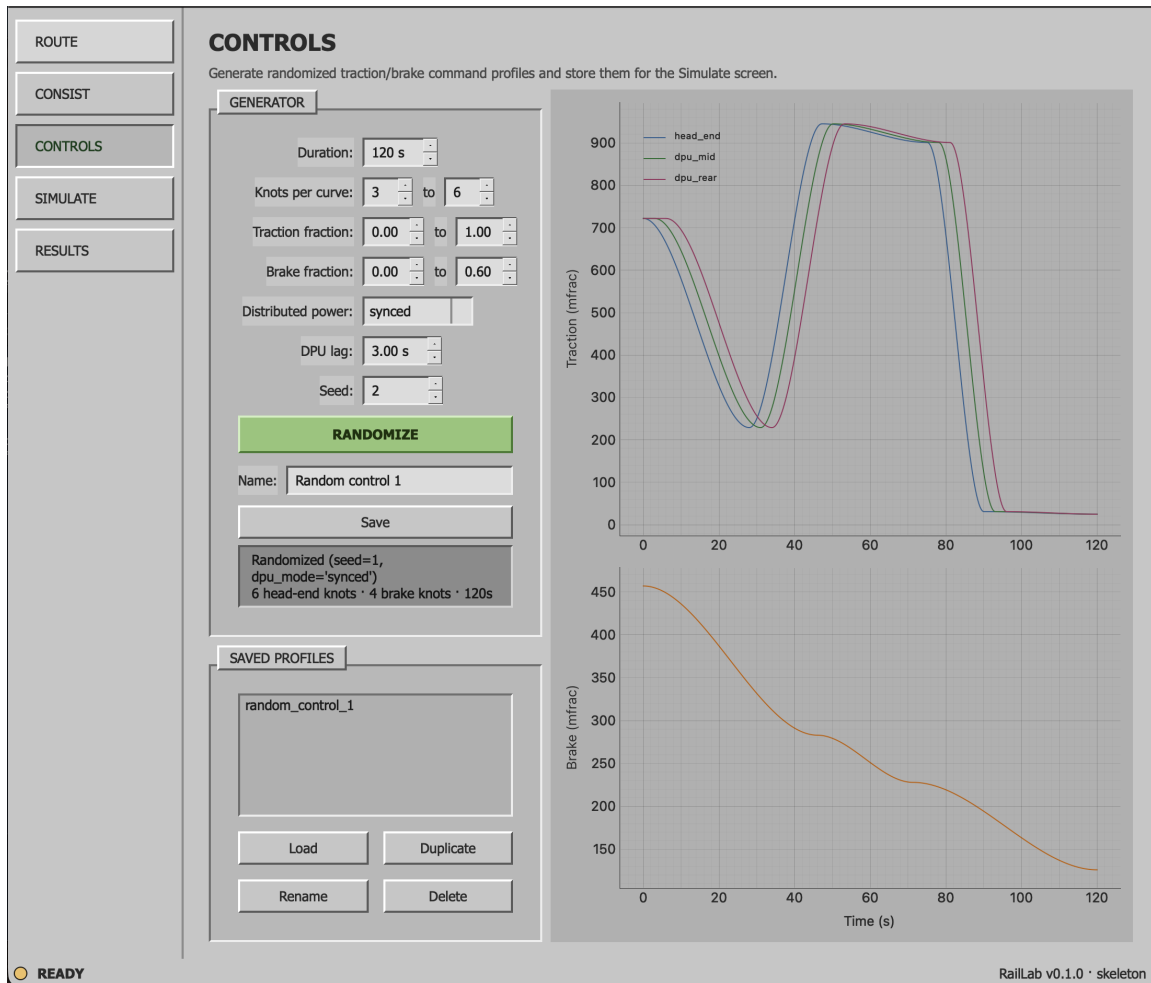


Figure 6.3: The **CONTROLS** screen: randomization bounds and distributed-power settings on the left, with a live preview of the generated traction command per power role (top right) and the shared brake command (bottom right). The offset between the three traction traces is the configured lag of the mid-train and rear helper units behind the head end.

Randomized commands are what the surrogate’s training set needs: a model trained only on hand-designed, sensible driving would never see the transients that make coupler dynamics hard. The bounded ranges keep the randomness within a plausible envelope rather than making it arbitrary.

6.3.4 Simulate and Results

The simulate screen is deliberately spare. It selects a saved train and a route, sets duration, an optional traction ceiling override, and the curvature-term scale, picks a control profile or falls back to the default ramp, and runs. Everything it needs has already been named and saved elsewhere, which is the point: a run is fully specified by a short list of named artifacts and four numbers, and is therefore trivially repeatable.

Results are plotted as they arrive: every vehicle’s speed on one axis and every coupler’s force on a second, time-linked axis below it (fig. 6.4). Plotting every vehicle rather than a summary is intentional. The figure shows a 126-vehicle consist over 120 seconds under a randomized control profile, and the structure that carries the information is the *spread* between vehicles: the envelope of the speed traces widening and collapsing as slack runs in and out, and the corresponding fan of coupler forces. Neither is visible in a mean, and the qualitative behavior, oscillation that decays rather than growing, forces that stay bounded, is the sanity check that a summary statistic cannot provide.

6.4 Status and limitations

RailLab is a working instrument, not finished software, and several limitations bear on how it should be read.

Runs cannot be interrupted or monitored mid-flight, for the reason given in section 6.2: the solver call is atomic. Long runs therefore occupy the application until they complete, though not the interface thread. The results screen plots the two channels that matter most for the present work and does not yet expose the full node and edge state, nor does it export a run directly; dataset generation remains the job of the headless batch path, which is the correct division, since the interface is for looking at one run and the batch runner is for producing thousands.

Most importantly, nothing in this chapter changes the standing caveat from chapter 4: the parameters in the car library are realistic in shape but are not calibrated to any specific real rolling stock. The application makes it easy to build a train and watch it behave, and that ease should not be mistaken for validation. What is being explored is the reference simulator’s behavior, which is exactly what the surrogate of the following chapters is trained to reproduce.

6.5 Chapter summary

This chapter documented RailLab, the desktop console built over the reference simulator and route pipeline. Its architecture keeps the interface strictly separated from the numerical backend through a service layer, so the backend remains headless and testable and later

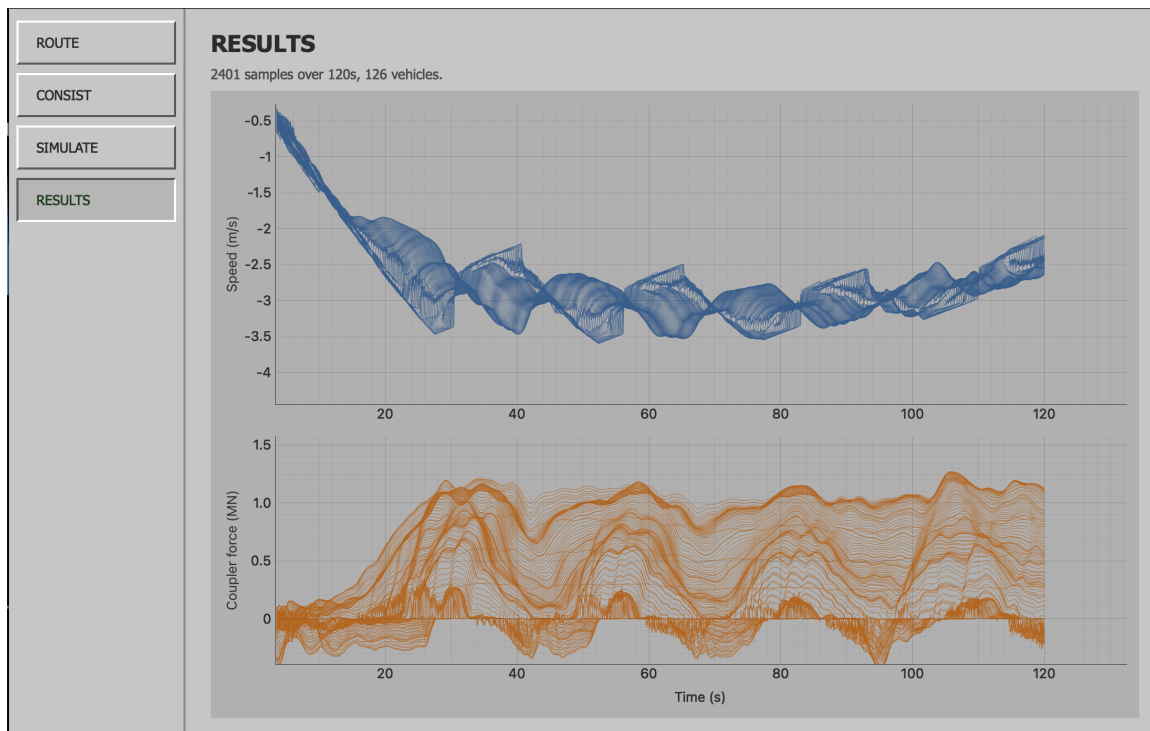


Figure 6.4: The RESULTS screen after a 120 s run of a 126-vehicle consist under a randomized control profile. Every vehicle's speed is plotted above every coupler's force, on time-linked axes. The information is carried by the spread between traces: the widening and collapsing speed envelope is slack running in and out, with the corresponding fan of coupler forces below.

components attach without disturbing existing screens. Its five screens follow the workflow the thesis needs: inspect a route, assemble a consist from a reusable car library, generate randomized driving commands, run, and look at the result. The artifacts it manipulates, car types, trains, control profiles, routes, are the same named, serializable objects the headless batch tooling uses, so interactive exploration and large-scale dataset generation share one representation. The next chapter uses that representation to build and train the surrogate model.